

DACF Project

AWS Capstone Project Report

Vighnesh, John and Anand

Engineering Science, Ngee Ann Polytechnic

July 2026

Abstract

This project implements a serverless data pipeline on AWS Cloud. Data was extracted from a HTTP endpoint, then stored in a S3 data-source bucket. The raw data was transformed to Parquet format using Python pandas library and catalogued with AWS Glue. AWS Athena was used for querying data, and quality checks were applied to ensure accuracy. The data is loaded into a Jupyter notebook for analytics in Python. The code is available in this [link](#).

Table of Contents

Abstract	2
1 Introduction and Objectives	4
2 Development Environment	6
3 AWS Glue Crawler and file queries with Athena	16
4 Additional Insights and Visualisations	27
6 Difficulties and Solutions	39
7 Reflection	40
8 AI Tools and Prompts	41
9 References	43

1 Introduction and Objectives

An organisation wants its analysts to be able to report on fishing impact in the open seas, and the infrastructure to support that reporting does not yet exist. Our task was to build that infrastructure inside our own AWS account and then prove it works by running real analytical queries through it. This capstone project builds an end-to-end AWS data engineering pipeline for analyzing global fisheries catch data (1950–2018) from the Sea Around Us dataset.

1.2 Dataset

The data comes from the Sea Around Us project, which publishes reconstructed global fisheries catch records covering 1950 to 2018 (Sea Around Us, n.d.). Therefore, this project supplements the estimates of unreported, discarded and small-scale catch. The whole dataset in Table 1) is partitioned into two sections:

- **Open seas (high seas)** ocean at least 200 nautical miles from any country's shoreline. Fish there are generally accepted as belonging to no single country.
- **Exclusive Economic Zone (EEZ)** ocean within 200 nautical miles of a country's shoreline, where that country typically claims exclusive access to the resources within it.

The dataset includes columns of information for each year, including which countries caught which types of fish in which areas (Sea Around Us, n.d.). The data also indicates how many tonnes of fish were caught and what the value of the catch was measured in 2010 US dollars (Sea Around Us, n.d.).

Table 1) Three source files that were used as part of this capstone

File	Scope	Rows / notes
SAU-GLOBAL-1-v48-0.csv	All open seas areas globally	561,675 lines including the header
SAU-HighSeas-71-v48-0.csv	One open seas area <i>Pacific, Western Central</i>	Adds area_name, species and commercial-group columns

File	Scope	Rows / notes
SAU-EEZ-242-v48-0.csv	The EEZ of one country Fiji	Adds data_layer and uncertainty_score; two columns named differently

1.3 Objectives

Table 2) Project objective versus what was achieved

Objectives	What was achieved
Launched and configured an AWS Cloud9 IDE instance	CapstoneIDE, t2.micro, deployed into the Capstone VPC public subnet; used as a working environment for all downloads, inspection and file conversion in Python CLI
Ran a SQL query against a single file using Amazon S3 Select	returned ten rows of fishing_entity, gear_type and tonnes directly from the object in S3
Transformed the CSV data files to Apache Parquet	using pandas with the pyarrow and fastparquet, and uploaded them to the data-source-#### S3 bucket via the AWS CLI
Created an AWS Glue crawler	fishcrawler, running under CapstoneGlueRole, output to database fishdb
Use Amazon Athena to query the data	confirming through a SELECT DISTINCT area_name query that the crawler had categorised rows from all three source files into one coherent table
Create an Athena view	Athena view for the Individual Challenge query, and the MackerelsCatch view used for visualisation

Beyond the six objectives, we were tasked to take the three query results set out of Athena and into set out of Athena and into Python, where they could be joined with derived quantities, tested for significance, and checked for outliers, rather than simply displayed as returned.

1.4 AWS Services Used

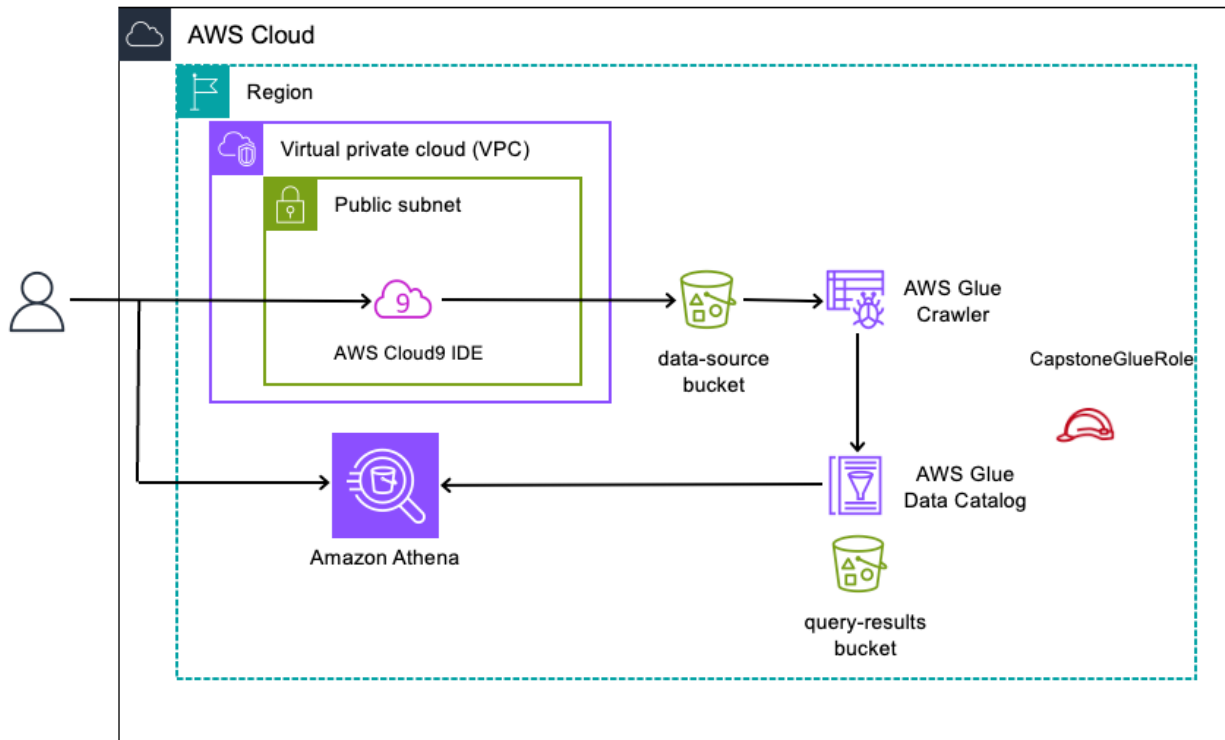


Figure 1) The overall architecture of the capstone project

2 Development Environment

AWS Cloud9 served as a cloud-based IDE for developing and running scripts. From the Cloud9 terminal, the team downloaded the source CSV files (Global, HighSeas, and EEZ datasets) and used pandas/pyarrow in a Python shell to convert them to Parquet format for efficient storage and querying.

This section describes the steps taken to configure the development environment, covering verification of IAM permissions, provisioning of the Cloud9 IDE, creation of S3 storage buckets, and retrieval of the source datasets.

2.1 Configuring the development environment

First step is to create the Capstone IDE for Cloud 9. This can be seen from Figure 2.1) and Figure 2.2).

Create environment [Info](#)

Details

Name

CapstoneIDE

Limit of 60 characters, alphanumeric and unique per user.

Description – optional

Limit 200 characters.

Environment type [Info](#)

Determines what the Cloud9 IDE will run on.

☒ New EC2 instance

Cloud9 creates an EC2 instance in your account. The configuration of your EC2 instance cannot be changed by Cloud9 after creation.

☐ Existing compute

You have an existing instance or server that you'd like to use.

New EC2 instance

Instance type [Info](#)

The memory and CPU of the EC2 instance that will be created for Cloud9 to run on.

☒ t2.micro (1 GiB RAM + 1 vCPU)

Free-tier eligible. Ideal for educational users and exploration.

☐ t3.small (2 GiB RAM + 2 vCPU)

Recommended for small web projects.

☐ m5.large (8 GiB RAM + 2 vCPU)

Recommended for production and most general-purpose development.

☐ Additional instance types

Explore additional instances to fit your needs.

Figure 2.1) Naming of IDE, Environment type and Instance type

Network settings [Info](#)

Connection

How your environment is accessed.

☒ AWS Systems Manager (SSM)

Accesses environment via SSM without opening inbound ports (no ingress).

☐ Secure Shell (SSH)

Accesses environment directly via SSH, opens inbound ports.

▼ VPC settings [Info](#)

Amazon Virtual Private Cloud (VPC)

The VPC that your environment will access. To allow the AWS Cloud9 environment to connect to its EC2 instance, attach an Internet gateway (IGW) to your VPC. [Create new VPC](#)

vpc-082454321c2c0c94f

Name –

Subnet

Used to setup your VPC configuration. To use a private subnet, select AWS Systems Manager (SSM) as the connection type. [Create new subnet](#)

No preference

Uses default subnet in any Availability Zone

► Tags – optional [Info](#)

A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.

Figure 2.2) VPC and Network Settings

Due to having issues with the Capstone Lab Environment. AWS System Manager (SSM) was used instead of Secure Shell (SSH). The environment used was AWS Learners Lab.

2.2 S3 data-source and query-results bucket

Two buckets were created in the us-east-1 region, with all settings other than the name left at their defaults. The steps to creating data-source bucket is seen in Figure 2); the same steps apply to creation of the query-results bucket.

Table 4) data-source-#### and query-results-#### buckets and their purposes

Bucket	Purpose
data-source-####	Holds the Parquet source objects that the Glue crawler reads and Athena queries
query-results-####	Holds the output Athena writes for every query executed

General configuration

AWS Region
US East (N. Virginia) us-east-1

Bucket type [Info](#)

☒ **General purpose**
Recommended for most use cases and access patterns. General purpose buckets are the original S3 bucket type. They allow a mix of storage classes that redundantly store objects across multiple Availability Zones.

☐ **Directory**
Recommended for low-latency use cases. These buckets use only the S3 Express One Zone storage class, which provides faster processing of data within a single Availability Zone.

Bucket namespace
Choose the namespace where you want to create your bucket. [Learn more](#)

☒ **Global namespace**
By default, S3 creates general purpose buckets in the global namespace.

☐ **Account Regional namespace (recommended)**
General purpose buckets created in your account Regional namespace are unique to your account. These buckets can never be created by another AWS account.

Bucket name [Info](#)

data-source-ps3208a

Bucket names must be 3 to 63 characters and unique within the global namespace. Bucket names must also begin and end with a letter or number. Valid characters are a-z, 0-9, periods (.), and hyphens (-). [Learn more](#)

Copy settings from existing bucket - optional
Only the bucket settings in the following configuration are copied.

[Choose bucket](#)

Format: s3://bucket/prefix

Figure 2.3) AWS screen of data-source bucket configuration

2.3 Downloading and inspecting the CSV files

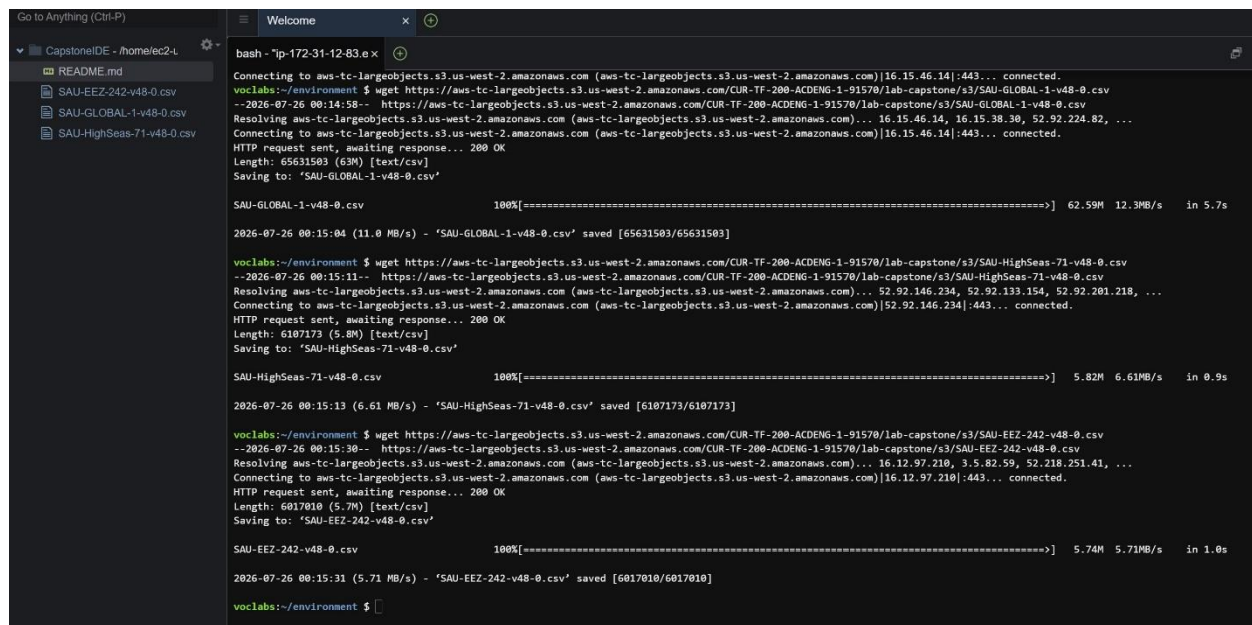
We downloaded the data that we will be using for the Capstone Project into the Cloud9 environment with `wget`. The following will be downloaded through the Capstone IDE and using the following terminal commands:


```
wget https://aws-tc-largeobjects.s3.us-west-2.amazonaws.com/CUR-TF-200-ACDENG-1-91570/lab-capstone/s3/SAU-GLOBAL-1-v48-0.csv
```

```
wget https://aws-tc-largeobjects.s3.us-west-2.amazonaws.com/CUR-TF-200-ACDENG-1-91570/lab-capstone/s3/SAU-HighSeas-71-v48-0.csv
```

```
wget https://aws-tc-largeobjects.s3.us-west-2.amazonaws.com/CUR-TF-200-ACDENG-1-91570/lab-capstone/s3/SAU-EEZ-242-v48-0.csv
```

The files can be seen having been successfully downloaded on the left panel in Figure 2.4).



```
bash - "ip-172-31-12-83.ex"
Connecting to aws-tc-largeobjects.s3.us-west-2.amazonaws.com (aws-tc-largeobjects.s3.us-west-2.amazonaws.com)[16.15.46.14]:443... connected.
voclabs:~/environment $ wget https://aws-tc-largeobjects.s3.us-west-2.amazonaws.com/CUR-TF-200-ACDENG-1-91570/lab-capstone/s3/SAU-GLOBAL-1-v48-0.csv
--2026-07-26 00:14:58-- https://aws-tc-largeobjects.s3.us-west-2.amazonaws.com/CUR-TF-200-ACDENG-1-91570/lab-capstone/s3/SAU-GLOBAL-1-v48-0.csv
Resolving aws-tc-largeobjects.s3.us-west-2.amazonaws.com (aws-tc-largeobjects.s3.us-west-2.amazonaws.com)... 16.15.46.14, 16.15.38.30, 52.92.224.82, ...
Connecting to aws-tc-largeobjects.s3.us-west-2.amazonaws.com (aws-tc-largeobjects.s3.us-west-2.amazonaws.com)[16.15.46.14]:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 65631503 (63M) [text/csv]
Saving to: 'SAU-GLOBAL-1-v48-0.csv'

SAU-GLOBAL-1-v48-0.csv      100%[=====] 62.59M  12.3MB/s  in 5.7s

2026-07-26 00:15:04 (11.0 MB/s) - 'SAU-GLOBAL-1-v48-0.csv' saved [65631503/65631503]

voclabs:~/environment $ wget https://aws-tc-largeobjects.s3.us-west-2.amazonaws.com/CUR-TF-200-ACDENG-1-91570/lab-capstone/s3/SAU-HighSeas-71-v48-0.csv
--2026-07-26 00:15:11-- https://aws-tc-largeobjects.s3.us-west-2.amazonaws.com/CUR-TF-200-ACDENG-1-91570/lab-capstone/s3/SAU-HighSeas-71-v48-0.csv
Resolving aws-tc-largeobjects.s3.us-west-2.amazonaws.com (aws-tc-largeobjects.s3.us-west-2.amazonaws.com)... 52.92.146.234, 52.92.133.154, 52.92.201.218, ...
Connecting to aws-tc-largeobjects.s3.us-west-2.amazonaws.com (aws-tc-largeobjects.s3.us-west-2.amazonaws.com)[52.92.146.234]:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 6107173 (5.8M) [text/csv]
Saving to: 'SAU-HighSeas-71-v48-0.csv'

SAU-HighSeas-71-v48-0.csv  100%[=====] 5.82M  6.61MB/s  in 0.9s

2026-07-26 00:15:13 (6.61 MB/s) - 'SAU-HighSeas-71-v48-0.csv' saved [6107173/6107173]

voclabs:~/environment $ wget https://aws-tc-largeobjects.s3.us-west-2.amazonaws.com/CUR-TF-200-ACDENG-1-91570/lab-capstone/s3/SAU-EEZ-242-v48-0.csv
--2026-07-26 00:15:30-- https://aws-tc-largeobjects.s3.us-west-2.amazonaws.com/CUR-TF-200-ACDENG-1-91570/lab-capstone/s3/SAU-EEZ-242-v48-0.csv
Resolving aws-tc-largeobjects.s3.us-west-2.amazonaws.com (aws-tc-largeobjects.s3.us-west-2.amazonaws.com)... 16.12.97.210, 3.5.82.59, 52.218.251.41, ...
Connecting to aws-tc-largeobjects.s3.us-west-2.amazonaws.com (aws-tc-largeobjects.s3.us-west-2.amazonaws.com)[16.12.97.210]:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 6017010 (5.7M) [text/csv]
Saving to: 'SAU-EEZ-242-v48-0.csv'

SAU-EEZ-242-v48-0.csv      100%[=====] 5.74M  5.71MB/s  in 1.0s

2026-07-26 00:15:31 (5.71 MB/s) - 'SAU-EEZ-242-v48-0.csv' saved [6017010/6017010]

voclabs:~/environment $
```

Figure 2.4) Using Terminal Commands to download respective csv files

We observed the column header row and first five rows of data in SAU-GLOBAL-1-v48-0.csv using:

```
head -6 SAU-GLOBAL-1-v48-0.csv
```

The following output is obtained:

```
year,fishing_entity,fishing_sector,catch_type,reporting_status,gear_type,end_use_t  
ype,tonnes,landed_value
```

```
1950,"Albania",Industrial,Discards,Unreported,bottom trawl,Direct human  
consumption,145.0013231580,212571.939749628
```

```
1950,"Albania",Industrial,Discards,Unreported,pelagic trawl,Direct human  
consumption,0.9863122230,1445.9337189179998
```

```
1950,"Albania",Industrial,Discards,Unreported,purse seine,Direct human  
consumption,0.4728285250,693.16661765
```

```
1950,"Albania",Industrial,Landings,Reported,bottom trawl,Direct human  
consumption,742.8529325409,1089022.3991050033
```

```
1950,"Albania",Industrial,Landings,Reported,bottom  
trawl,Other,0.7435965291,1090.11251161662
```

Among other details, each line of data in this dataset includes: the year that the fishing occurred the country (`fishing_entity`) that did the fishing, the tonnes of fish caught that year by that country and, the value in 2010 US dollars (`landed_value`) of the fish caught that year by that country.

When we ran `wc -l SAU-GLOBAL-1-v48-0.csv` it returns no of lines in dataset 561675 `SAU-GLOBAL-1-v48-0.csv`. The dataset includes reported and "best guess" data for all fishing that occurred in the global high seas (meaning, *not* in any one country's EEZ) between 1950 and 2018.

2.5 Converting the CSV files to Parquet

Now we have to convert the files to parquet. This can be done by following steps:

1. Install the required libraries in the Cloud9 terminal:

```
sudo pip3 install pandas pyarrow fastparquet
```

2. Start the Python interactive shell:

```
python3
```

3. Import Pandas:

```
import pandas as pd
```

4. Read the CSV file into a DataFrame:

```
df = pd.read_csv('<file_name>.csv')
```

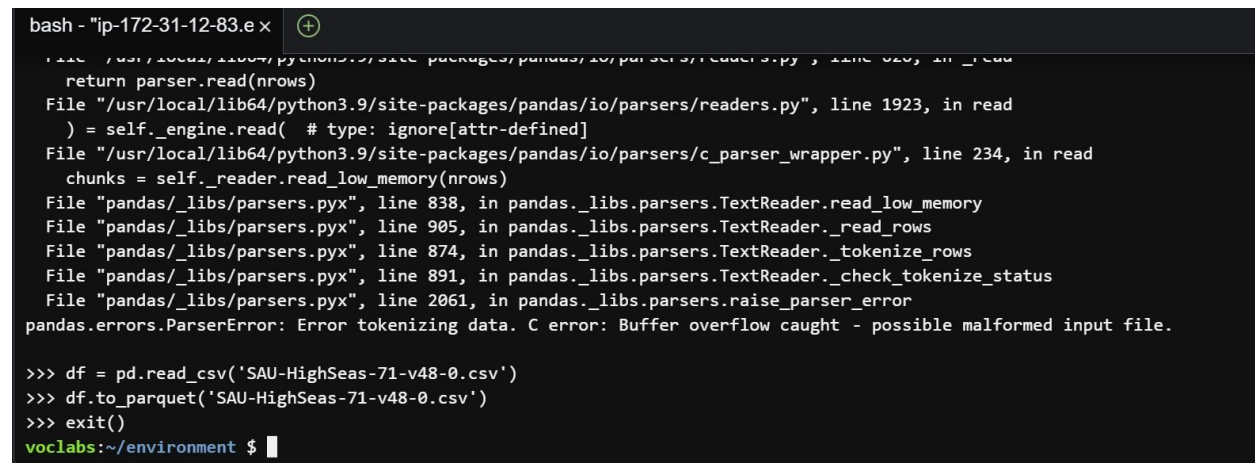
5. Convert the DataFrame to Parquet:

```
df.to_parquet('<file_name>.parquet', index=False)
```

6. Exit the Python interactive shell:

```
exit()
```

The command line output can be seen in Figure 2.5) and the result in the IDE left panel can be seen in Figure 2.6).



```
bash - "ip-172-31-12-83.e x"
... /usr/local/lib64/python3.9/site-packages/pandas/io/parsers/readers.py", line 620, in _read
    return parser.read(nrows)
File "/usr/local/lib64/python3.9/site-packages/pandas/io/parsers/readers.py", line 1923, in read
    ) = self._engine.read( # type: ignore[attr-defined]
File "/usr/local/lib64/python3.9/site-packages/pandas/io/parsers/c_parser_wrapper.py", line 234, in read
    chunks = self._reader.read_low_memory(nrows)
File "pandas/_libs/parsers.pyx", line 838, in pandas._libs.parsers.TextReader.read_low_memory
File "pandas/_libs/parsers.pyx", line 905, in pandas._libs.parsers.TextReader._read_rows
File "pandas/_libs/parsers.pyx", line 874, in pandas._libs.parsers.TextReader._tokenize_rows
File "pandas/_libs/parsers.pyx", line 891, in pandas._libs.parsers.TextReader._check_tokenize_status
File "pandas/_libs/parsers.pyx", line 2061, in pandas._libs.parsers.raise_parser_error
pandas.errors.ParserError: Error tokenizing data. C error: Buffer overflow caught - possible malformed input file.

>>> df = pd.read_csv('SAU-HighSeas-71-v48-0.csv')
>>> df.to_parquet('SAU-HighSeas-71-v48-0.csv')
>>> exit()
voclabs:~/environment $
```

Figure 2.5: Converting from csv to parquet

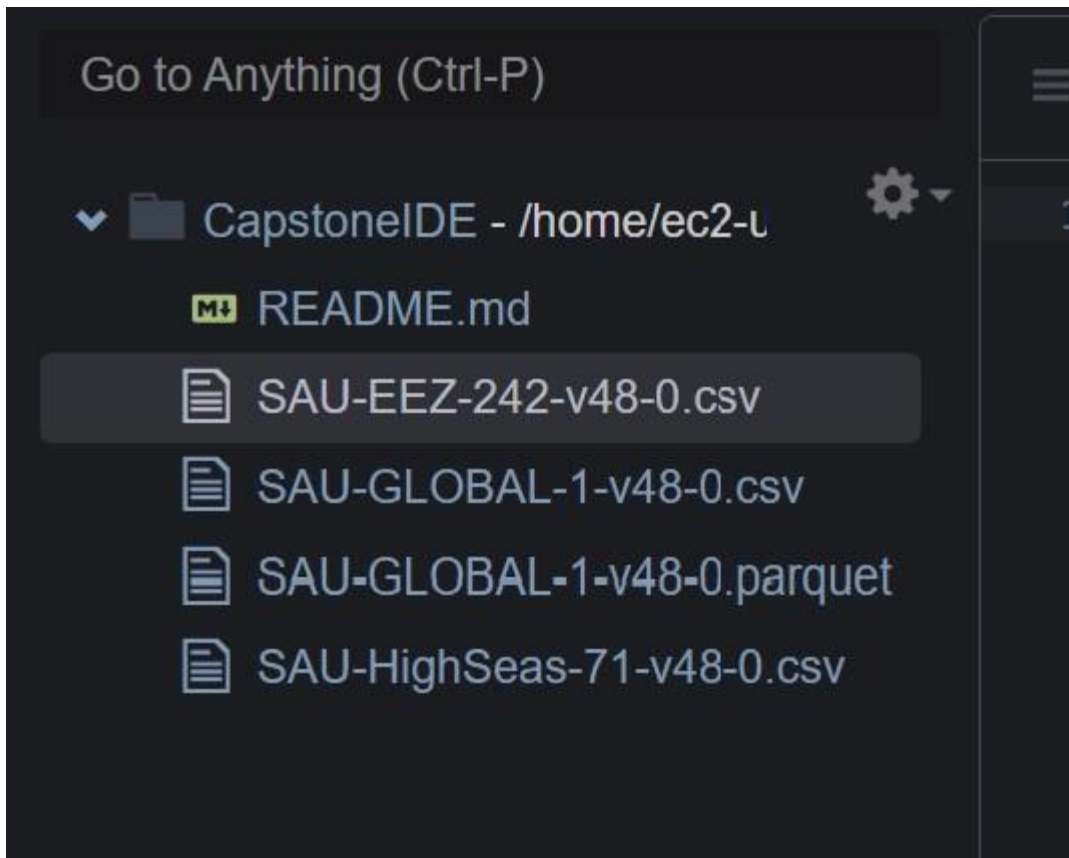


Figure 2.6: Global File has been successfully converted from CSV to Parquet

2.4 Reason for converting files from CSV to parquet

AWS Athena bills by data scanned, with .csv Athena typically has to read more data from the files even if you only select a few columns. However, Apache Parquet stores all values of each column contiguously. Thus, a query like `SELECT DISTINCT area_name` often outperforms the CSV file format as it does not need to read every byte of the file. The following section explains in greater detail Apache Parquet file format and its advantages:

1. Columnar Storage

Parquet organizes data by columns rather than rows, unlike row-based formats such as CSV.

- **Efficient Querying:** Queries that target specific columns can retrieve data faster, since only the required columns need to be read.

- **Better Compression:** Because similar data is grouped together within a column, it compresses far more effectively than mixed data stored row by row.

2. Schema Definition

Parquet files carry an embedded schema that defines the data's structure, types, and relationships.

- **Data Validation:** Enforces predefined data types, reducing processing errors.
- **Self-Descriptive:** Tools can interpret the file without needing an external schema definition.

3. Advanced Compression

Parquet supports multiple compression algorithms (Snappy, Gzip, Zstandard), which combine with its columnar format for highly efficient storage.

- **Reduced Storage Costs:** File sizes are significantly smaller than row-based formats — up to 10x smaller than CSV in some cases.
- **Faster Data Transfers:** Smaller files move across networks more quickly.

4. Predicate Pushdown

Parquet uses metadata and indexes to skip over irrelevant data during reads.

- **Performance Boost:** Minimizes unnecessary I/O operations, speeding up queries.
- **Resource Efficiency:** Reduces CPU and memory load during processing.

5. Rich Metadata

Parquet files include detailed metadata (field names, types, encoding/compression settings, row groups, and column chunks).

- **Data Insights:** Offers a quick overview of file structure and content.

- **Compatibility:** Enables seamless integration with data tools and frameworks.

6. Row Grouping

Data is organized into row groups, further split into column chunks.

- **Parallel Processing:** Distributed systems (e.g., Spark) can process row groups independently.
- **Scalability:** Large datasets are split into manageable chunks for easier handling.

7. Support for Nested Data Structures

Parquet natively handles complex data types like arrays, maps, and structs — unlike flat formats such as CSV.

- **Versatility:** Easier to store and analyze hierarchical or relational data.
- **No Flattening Required:** Avoids the need to transform complex data into flat tables.

8. Wide Ecosystem Support

Parquet is natively supported by major data processing frameworks (Spark, Hadoop, Hive, etc.).

- **Seamless Integration:** Fits naturally into modern data pipelines.
- **Cross-Platform Compatibility:** Accessible via Python, Java, R, and other languages.

9. Optimized for Big Data

Designed for distributed environments, leveraging parallelism and compression to handle massive datasets.

- **Use Cases:** Batch processing (Hadoop), interactive querying (Presto, Impala), and cloud analytics (AWS, BigQuery, Azure).

In a benchmark test, Parquet files were up to 5 times smaller and seven to ten times faster running analytical queries than CSV (Parquet vs CSV, 2025).]

2.6 Uploading the Parquet files to S3:

The final step is to upload the files to S3. This can be done so by using the following:

```
aws s3 cp SAU-HighSeas-71-v48-0.  
s3://data-source-ps3201/SAU-HighSeas-71-v48-0.parquet  
  
aws s3 cp SAU-HighSeas-71-v48-0.  
s3://data-source-ps3201/SAU-GLOBAL-1-v48-0.parquet  
  
aws s3 cp SAU-HighSeas-71-v48-0.  
s3://data-source-ps3201/SAU-EEZ-242-v48-0.parquet
```

```
voclabs:~/environment $ aws s3 cp SAU-HighSeas-71-v48-0.parquet s3://data-source-ps3208a/SAU-HighSeas-71-v48-0.parquet  
upload: ./SAU-HighSeas-71-v48-0.parquet to s3://data-source-ps3208a/SAU-HighSeas-71-v48-0.parquet  
voclabs:~/environment $ aws s3 cp SAU-HighSeas-71-v48-0.parquet s3://data-source-ps3208a/SAU-GLOBAL-1-v48-0.parquet  
upload: ./SAU-HighSeas-71-v48-0.parquet to s3://data-source-ps3208a/SAU-GLOBAL-1-v48-0.parquet  
voclabs:~/environment $ aws s3 cp SAU-HighSeas-71-v48-0.parquet s3://data-source-ps3208a/SAU-GLOBAL-1-v48-0.parquet  
upload: ./SAU-HighSeas-71-v48-0.parquet to s3://data-source-ps3208a/SAU-GLOBAL-1-v48-0.parquet  
voclabs:~/environment $
```

Figure 2.7: Uploading all 3 files to the S3 Bucket

[Objects](#)
[Metadata](#)
[Properties](#)
[Permissions](#)
[Metrics](#)
[Management](#)
[File systems - new](#)
[Access Points](#)

Objects (3)

[Refresh](#)
[Copy S3 URI](#)
[Copy URL](#)
[Download](#)
[Open](#)
[Delete](#)
[Actions](#)
[Create folder](#)
[Upload](#)

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

Find objects by prefix

☐	Name	Type	Last modified	Size	Storage class
☐	SAU-EEZ-242-v48-0.parquet	parquet	July 26, 2026, 10:40:24 (UTC+08:00)	559.4 KB	Standard
☐	SAU-GLOBAL-1-v48-0.parquet	parquet	July 26, 2026, 10:40:06 (UTC+08:00)	559.4 KB	Standard
☐	SAU-HighSeas-71-v48-0.parquet	parquet	July 26, 2026, 10:37:04 (UTC+08:00)	559.4 KB	Standard

Figure 2.8: Checking if the S3 Bucket has the files uploaded

In summary, we first configured AWS Cloud9 environment and created two buckets in S3. Then using Cloud9 terminal we downloaded the .csv files from source. We then used pandas to convert data to parquet format and finally uploaded the parquet file to S3 bucket.

3 AWS Glue Crawler and file queries with Athena

Task 1 ended with one file in S3 and a query tool that can only address one file at a time. Task 2 removes that limitation: a second data file is added, AWS Glue discovers the structure of both, and Athena queries them as a single table.

3.1 Workflow Overview

- Inspect and standardise the EEZ dataset.** The mismatched column names were identified and renamed with Pandas.
- Convert and upload the corrected file.** The corrected dataset was saved as CSV and Apache Parquet, then uploaded to the existing data-source S3 bucket.
- Refresh the metadata catalogue.** The AWS Glue crawler was rerun so the existing Athena table could recognise the newly added EEZ data.

4. **Validate and analyse the integrated dataset.** Athena queries were used to verify the new area value, compare Fiji catch values, create a view, and perform individual analysis.

3.2 Correcting and Converting the Fiji EEZ Dataset

Initial data inspection. The header of SAU-EEZ-242-v48-0.csv showed that two column names did not match the schema used by the Global and HighSeas files. The EEZ file used fish_name and country, while the existing dataset used common_name and fishing_entity. If these names were left unchanged, the Glue crawler could treat equivalent data as separate fields and Athena queries would not combine the records correctly.

```
head -1 SAU-EEZ-242-v48-0.csv
cp SAU-EEZ-242-v48-0.csv SAU-EEZ-242-v48-0-old.csv
```

Column correction with Pandas. A backup copy was loaded into a DataFrame. The two inconsistent columns were renamed, the updated column list was checked, and the corrected data was written to both CSV and Parquet formats.

```
import pandas as pd

data_location = 'SAU-EEZ-242-v48-0-old.csv'
df = pd.read_csv(data_location)

df.rename(columns={
    'fish_name': 'common_name',
    'country': 'fishing_entity'
}, inplace=True)

df.to_csv('SAU-EEZ-242-v48-0.csv', header=True, index=False)
df.to_parquet('SAU-EEZ-242-v48-0.parquet', index=False)
```

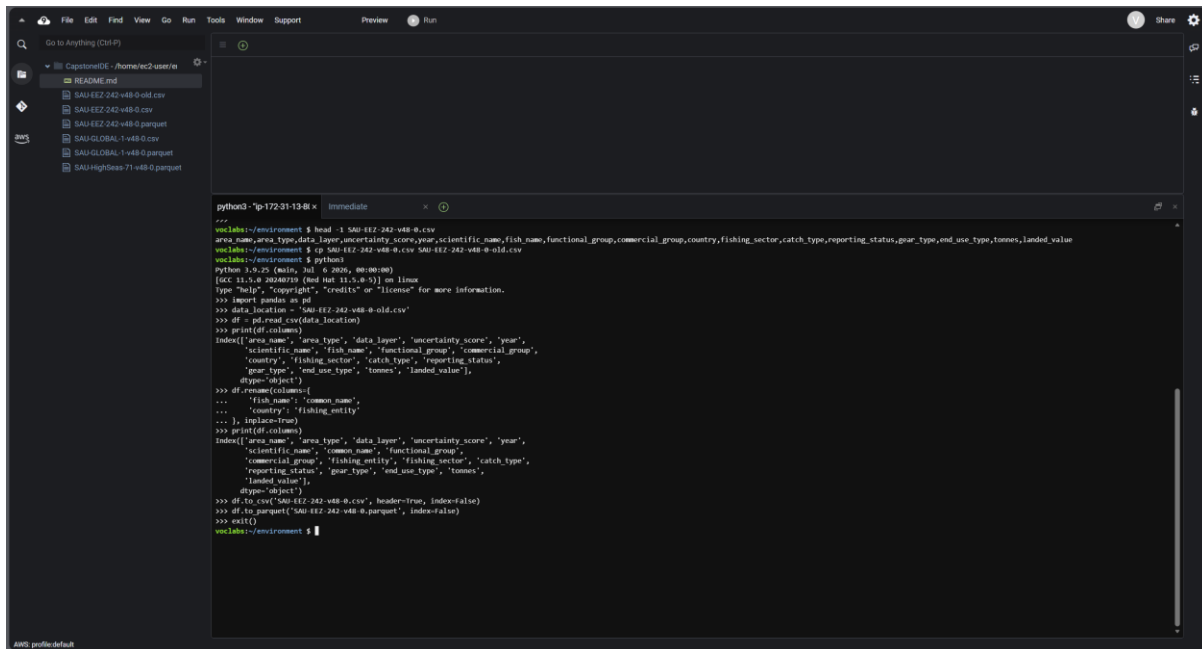


Figure 3.1. The original and corrected column names were checked before the updated CSV and Parquet files were created.

3.3 Uploading the File and Refreshing the Data Catalogue

S3 upload. The corrected Parquet file was uploaded to the same data-source bucket as the Global and HighSeas Parquet files. Keeping the analytical bucket restricted to Parquet files prevents Athena from attempting to interpret CSV output files as Parquet data.

```
aws s3 cp SAU-EEZ-242-v48-0.parquet \
s3://data-source-ps3201ak/SAU-EEZ-242-v48-0.parquet
```

```
aws - "ip-172-31-13-80.ec2" x Immediate x
voclabs:~/environment $ head -1 SAU-EEZ-242-v48-0.csv
area_name,area_type,data_layer,uncertainty_score,year,scientific_name,fish_name,functional_group,commercial_group,country,fishing_sector,catch_type,reporting_status,gear_type,end_use_type,tonnes,landed_value
voclabs:~/environment $ cp SAU-EEZ-242-v48-0.csv SAU-EEZ-242-v48-0-old.csv
voclabs:~/environment $ python3
Python 3.9.25 (main, Jul 6 2026, 00:00:00)
[GCC 11.5.0 20240719 (Red Hat 11.5.0-5)] on linux
Type "help", "copyright", "credits" or "license()" for more information.
>>> import pandas as pd
>>> data_location = 'SAU-EEZ-242-v48-0-old.csv'
>>> df = pd.read_csv(data_location)
>>> print(df.columns)
Index(['area_name', 'area_type', 'data_layer', 'uncertainty_score', 'year',
       'scientific_name', 'fish_name', 'functional_group', 'commercial_group',
       'country', 'fishing_sector', 'catch_type', 'reporting_status',
       'gear_type', 'end_use_type', 'tonnes', 'landed_value'],
      dtype='object')
>>> df.rename(columns={
...     'fish_name': 'common_name',
...     'country': 'fishing_entity'
... }, inplace=True)
>>> print(df.columns)
Index(['area_name', 'area_type', 'data_layer', 'uncertainty_score', 'year',
       'scientific_name', 'common_name', 'functional_group',
       'commercial_group', 'fishing_entity', 'fishing_sector', 'catch_type',
       'reporting_status', 'gear_type', 'end_use_type', 'tonnes',
       'landed_value'],
      dtype='object')
>>> df.to_csv('SAU-EEZ-242-v48-0.csv', header=True, index=False)
>>> df.to_parquet('SAU-EEZ-242-v48-0.parquet', index=False)
>>> exit()
voclabs:~/environment $ aws s3 cp SAU-EEZ-242-v48-0.parquet s3://data-source-ps3201ak/SAU-EEZ-242-v48-0.parquet
upload: ./SAU-EEZ-242-v48-0.parquet to s3://data-source-ps3201ak/SAU-EEZ-242-v48-0.parquet
voclabs:~/environment $
```

Figure 3.2. The corrected Fiji EEZ Parquet file was uploaded successfully using the AWS CLI.

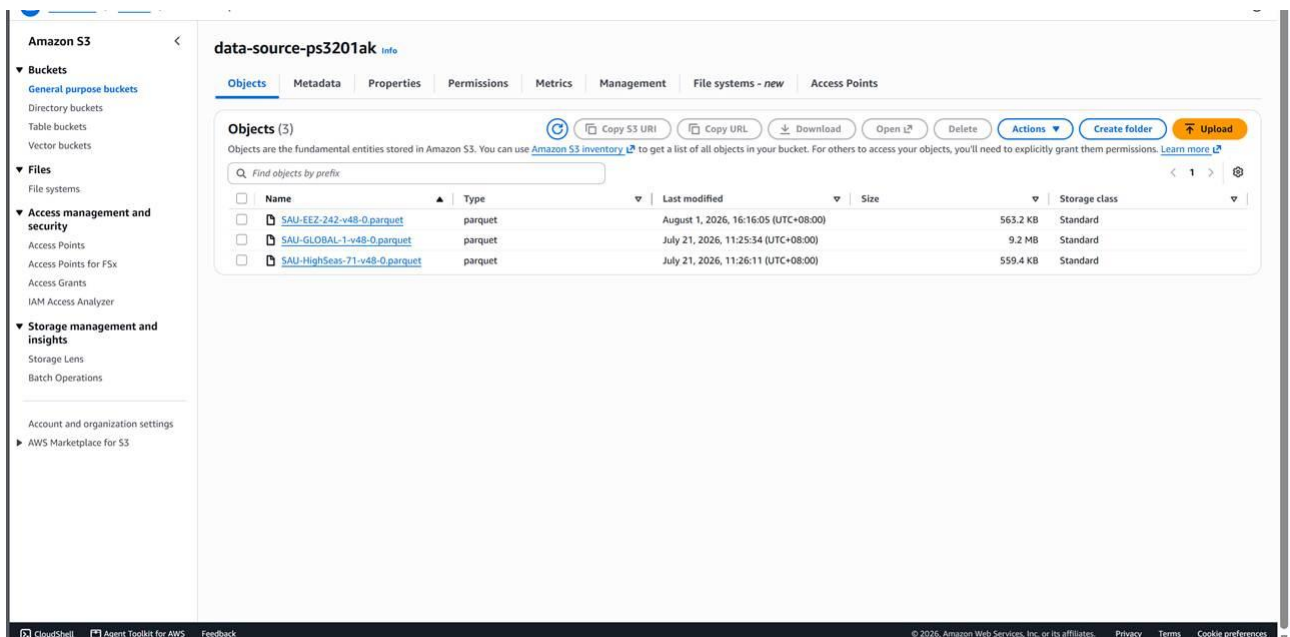


Figure 3.3. The data-source bucket contained the Global, HighSeas and corrected Fiji EEZ Parquet files.

Glue crawler update. After the upload, fishcrawler was rerun. This refreshed the metadata in fishdb so the existing table `data_source_ps3201ak` included the newly added EEZ records. The successful three-value `area_name` result shown below confirms that the new file was incorporated into the table.

3.4 Verifying the Integrated Dataset in Athena

Area verification. A DISTINCT query was used to check the area_name field. The result returned Fiji, Pacific, Western Central, and a blank value representing NULL. These three values correspond to the EEZ file, the regional HighSeas file, and the Global open-seas file respectively.

```
SELECT DISTINCT area_name
FROM fishdb.data_source_ps3201ak;
```

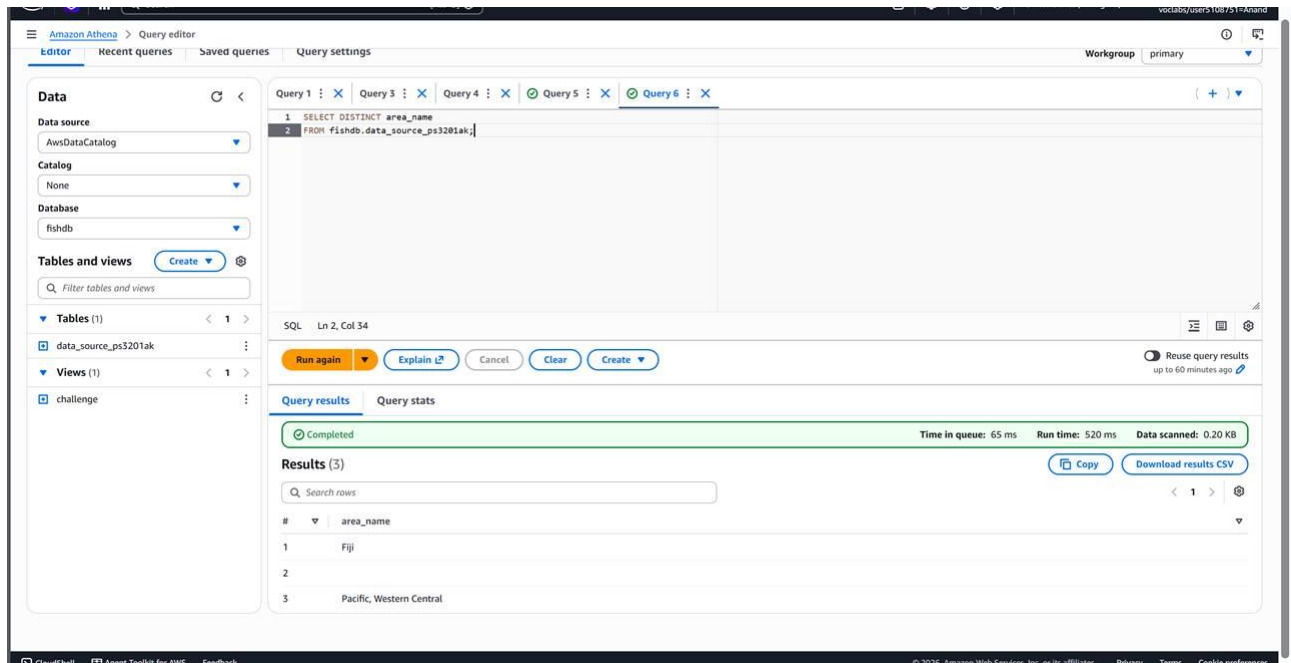


Figure 3.4. The three distinct area_name values confirmed that all three Parquet datasets were available through the Glue table.

3.5 Comparing Fiji Catch Values

Purpose of the analysis. Three queries were used to calculate the value of fish caught by Fiji from 2001 to 2018: open seas only, Fiji EEZ only, and both areas combined. The landed_value field was summed and cast to DECIMAL(38,2) so the results displayed as readable US dollar values instead of scientific notation.

3.5.1 Open-seas catch value

```
SELECT year, fishing_entity AS Country,
       CAST(CAST(SUM(landed_value) AS DOUBLE) AS DECIMAL(38,2))
       AS ValueOpenSeasCatch
```

```

FROM fishdb.data_source_ps3201ak
WHERE area_name IS NULL
      AND fishing_entity = 'Fiji'
      AND year > 2000
GROUP BY year, fishing_entity
ORDER BY year;

```

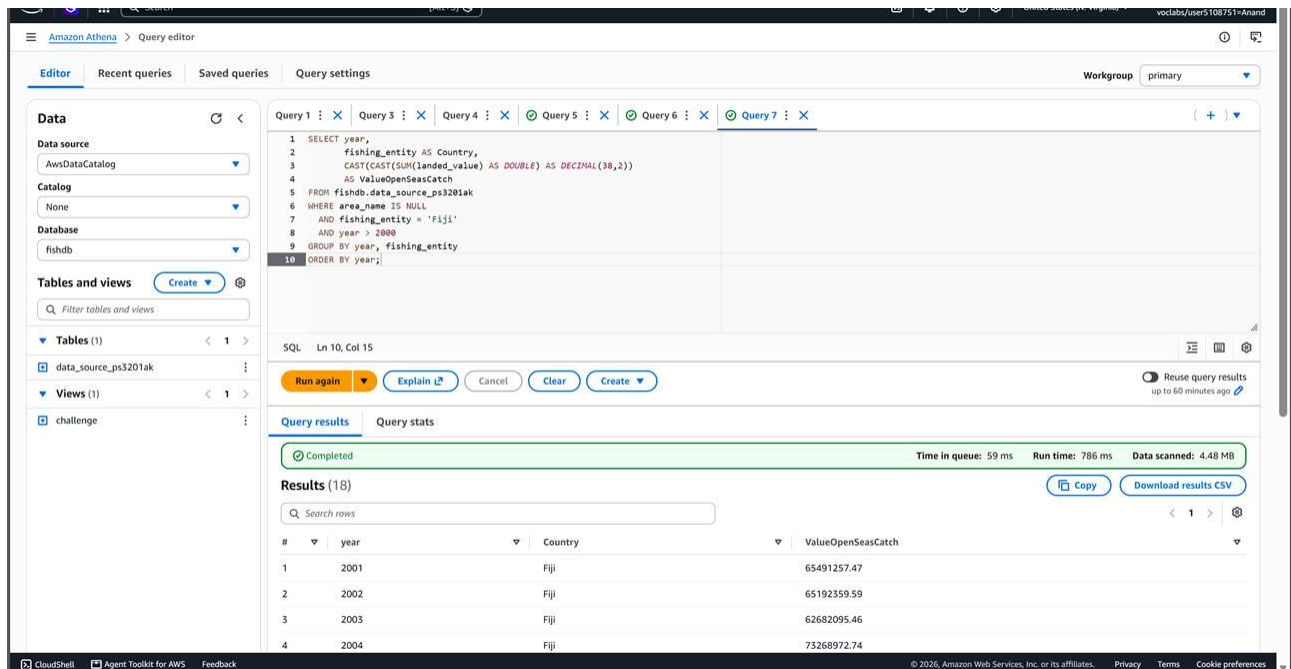


Figure 3.5. Fiji open-seas catch value by year.

3.5.2 Fiji EEZ catch value

```

SELECT year, fishing_entity AS Country,
       CAST(CAST(SUM(landed_value) AS DOUBLE) AS DECIMAL(38,2))
       AS ValueEEZCatch
FROM fishdb.data_source_ps3201ak
WHERE area_name LIKE '%Fiji%'
      AND fishing_entity = 'Fiji'
      AND year > 2000
GROUP BY year, fishing_entity
ORDER BY year;

```

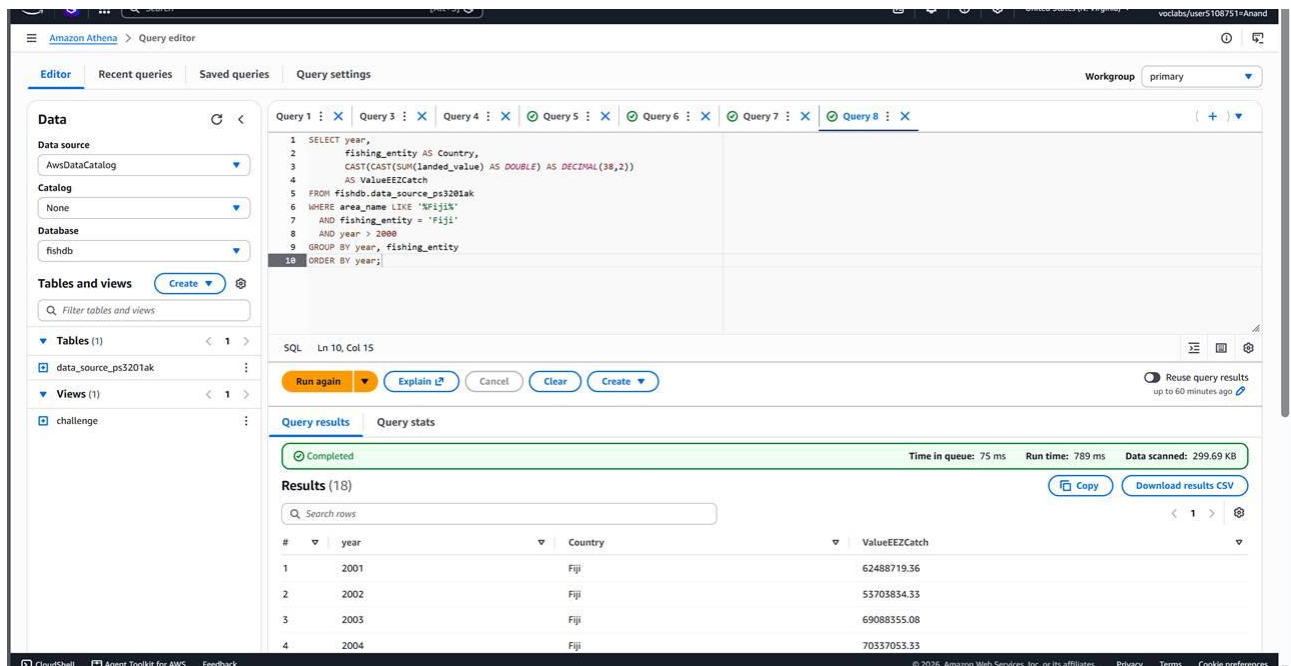


Figure 3.6. Fiji EEZ catch value by year.

3.5.3 Combined Fiji EEZ and open-seas catch value

```
SELECT year, fishing_entity AS Country,
       CAST(CAST(SUM(landed_value) AS DOUBLE) AS DECIMAL(38,2))
       AS ValueEEZAndOpenSeasCatch
FROM fishdb.data_source_ps3201ak
WHERE (area_name LIKE '%Fiji%' OR area_name IS NULL)
      AND fishing_entity = 'Fiji'
      AND year > 2000
GROUP BY year, fishing_entity
ORDER BY year;
```

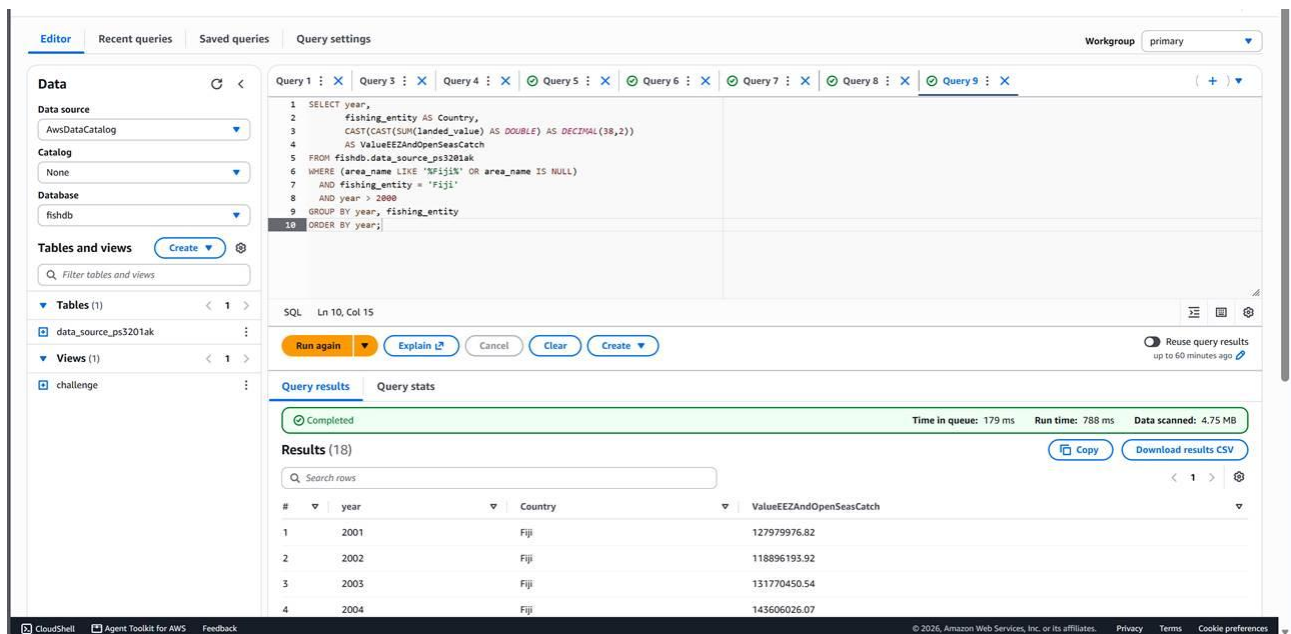


Figure 3.7. Combined Fiji EEZ and open-seas catch value by year.

3.5.4 Checkpoint validation

Year	Open seas (US\$)	Fiji EEZ (US\$)	Combined (US\$)
2001	65,491,257.47	62,488,719.36	127,979,976.82

Validation result

The 2001 values matched the project checkpoints. The displayed open-seas and EEZ values add to US\$127,979,976.83, while Athena returned US\$127,979,976.82 for the combined query. The one-cent difference is caused by rounding the displayed component results separately after the underlying values have already been aggregated.

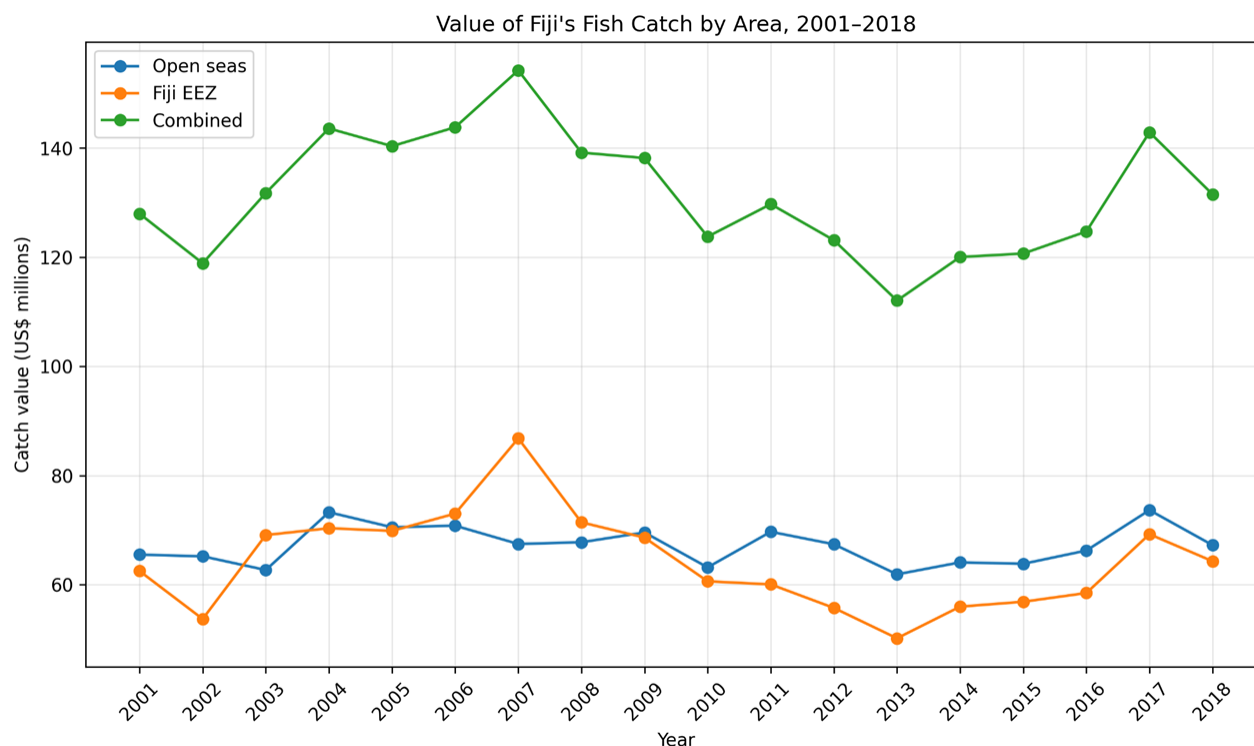


Figure 3.8. Comparison of Fiji open-seas, EEZ and combined catch values from 2001 to 2018.

3.5.5 Findings from the Fiji comparison

- The combined catch value was highest in 2007 at approximately US\$154.25 million and lowest in 2013 at approximately US\$112.06 million.
- Open-seas catch value exceeded Fiji EEZ catch value in 14 of the 18 years, showing that the open-seas component was generally the larger contributor during the period.
- Fiji EEZ catch value showed greater variation, rising to approximately US\$86.80 million in 2007 before falling to approximately US\$50.17 million in 2013.
- Both components recovered after 2013, with the combined value reaching approximately US\$142.90 million in 2017 before decreasing in 2018.

3.6 Creating and Querying the MackerelsCatch View

View creation. A reusable Athena view was created to isolate records who's common_name contained “Mackerels” for years after 2014. The view stored the year, area, country, and total weight so that further analysis could be performed without repeating the full filtering logic.

```
CREATE OR REPLACE VIEW MackerelsCatch AS
SELECT year, area_name AS WhereCaught,
       fishing_entity AS Country,
       SUM(tonnes) AS TotalWeight
FROM fishdb.data_source_ps3201ak
WHERE common_name LIKE '%Mackerels%'
      AND year > 2014
GROUP BY year, area_name, fishing_entity, tonnes
ORDER BY tonnes DESC;
```

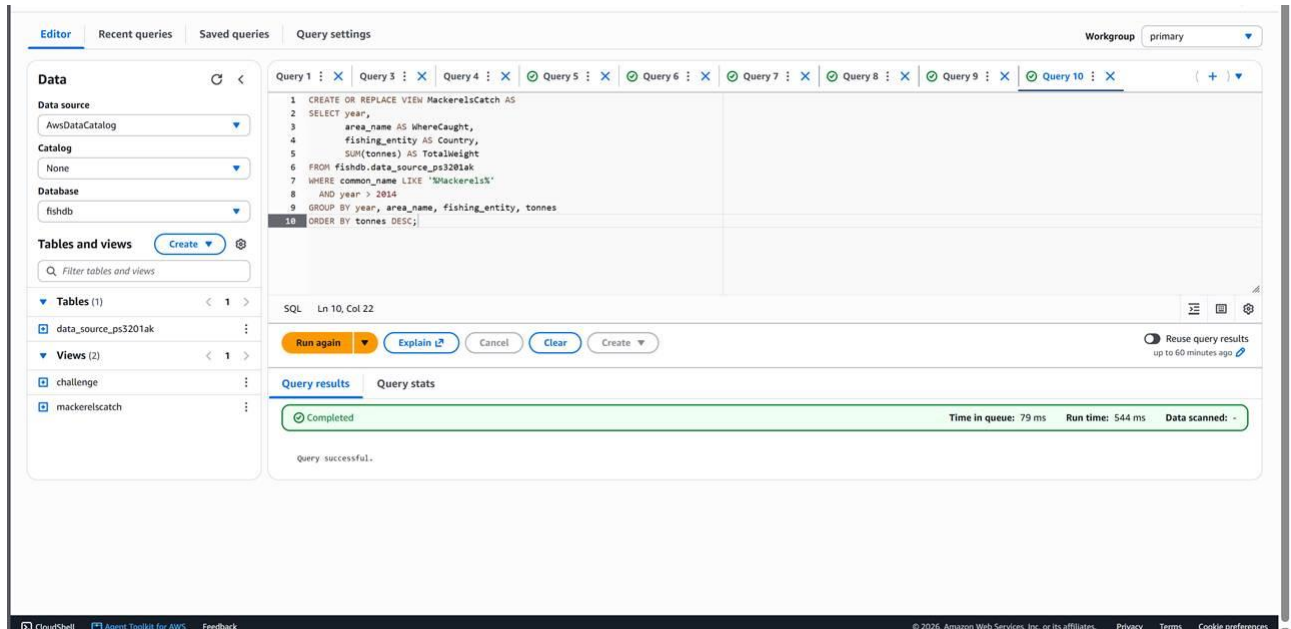


Figure 3.9. The MackerelsCatch view was created successfully in Athena.

Country and year analysis. The required query grouped the view by year and country and returned the maximum TotalWeight for each combination. The output

contained 78 rows, which is expected because multiple countries were returned for each year.

```
SELECT year, Country, MAX(TotalWeight) AS Weight
FROM fishdb.mackerelscatch
GROUP BY year, Country
ORDER BY year, Weight DESC;
```

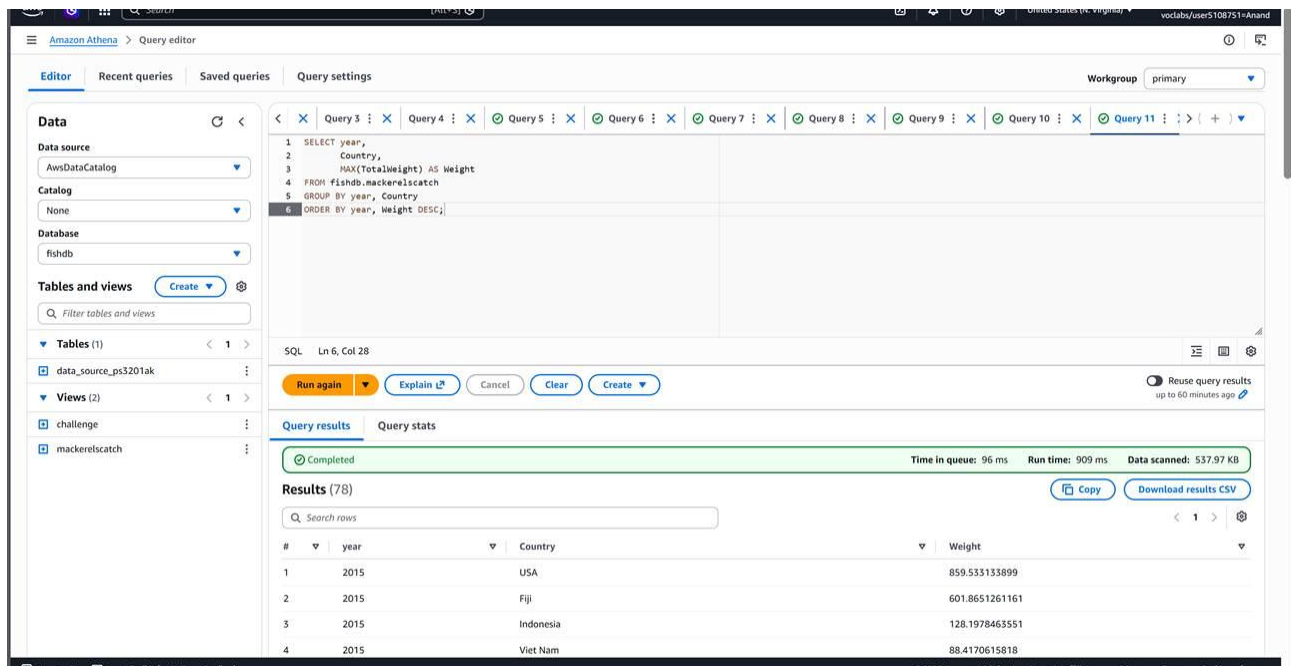


Figure 3.10. Mackerel catch by country and year; the Athena query returned 78 year-country combinations.

Required China filter. The view was then filtered to display only China records. This confirmed that the view could support country-specific analysis.

```
SELECT *
FROM fishdb.mackerelscatch
WHERE country IN ('China');
```

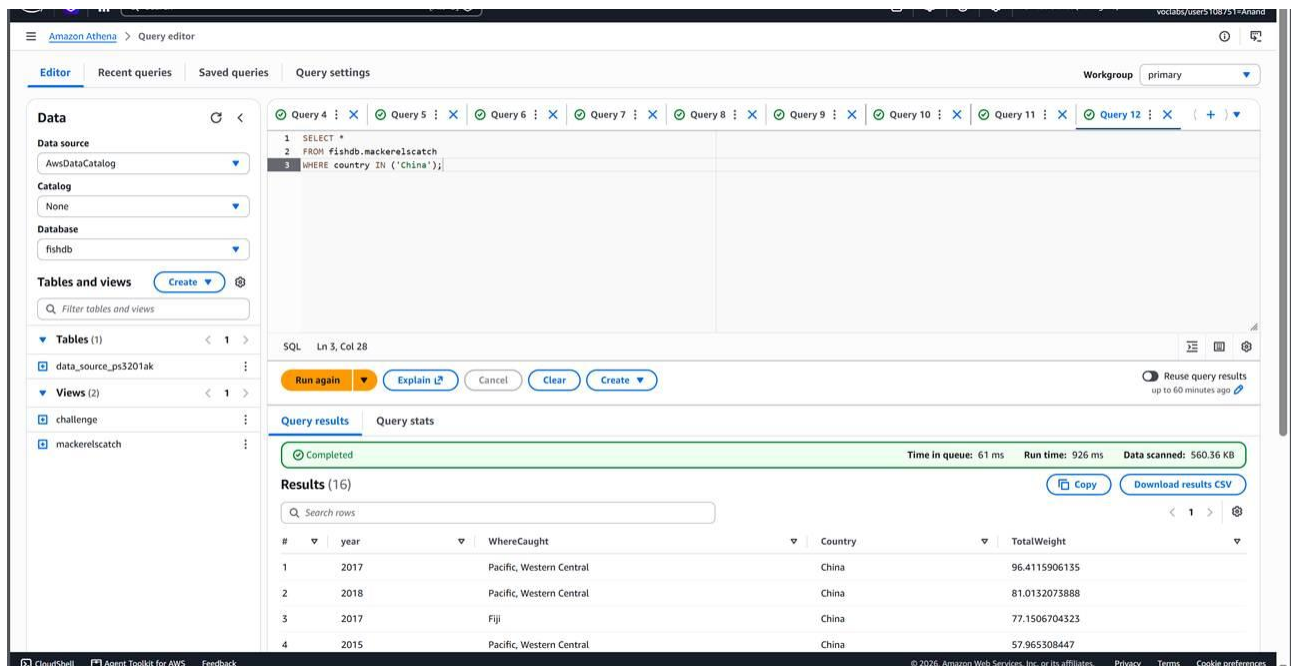


Figure 3.11. China records retrieved from the MackerelsCatch view.

4 Additional Insights and Visualisations

Rationale. Two additional queries were completed to exceed the minimum individual requirement and provide both time-series and cross-country insights. The second group member should include a separate individual query in the final combined report.

4.1 China's total mackerel catch by year

```
SELECT year,
       CAST(SUM(TotalWeight) AS DECIMAL(18,2))
       AS ChinaMackerelsCatch
FROM fishdb.mackerelscatch
WHERE Country = 'China'
GROUP BY year
ORDER BY year;
```

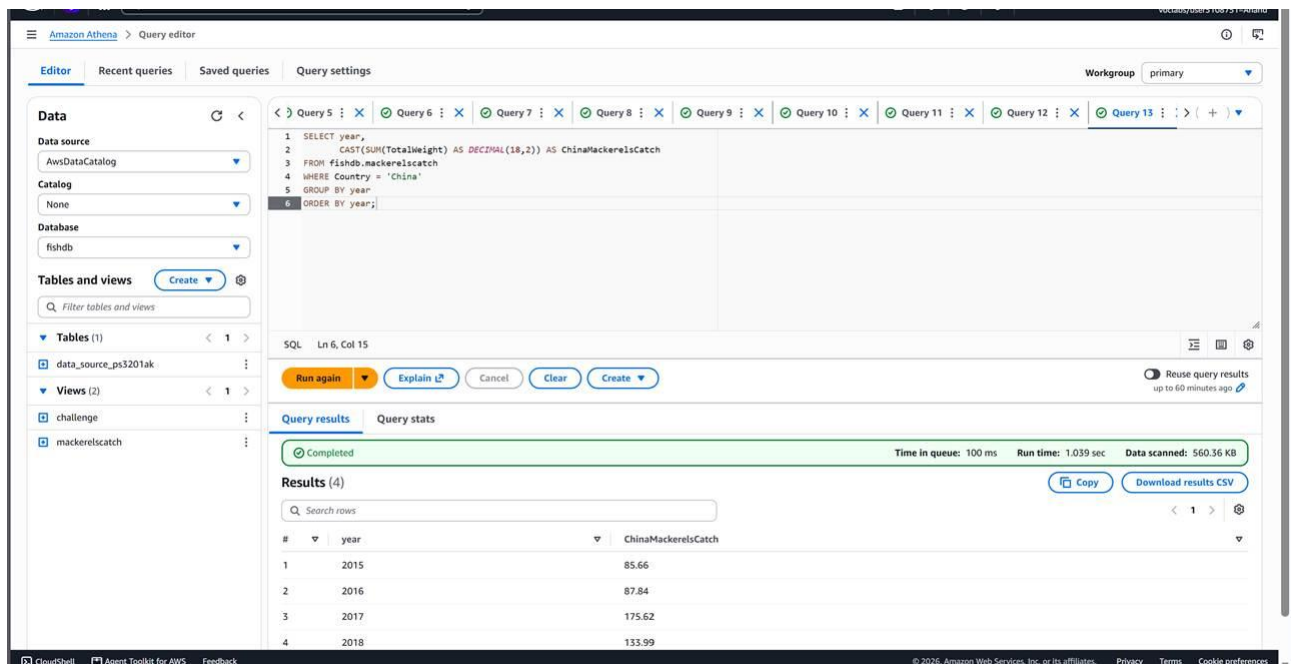


Figure 4.1.1 Individual query used to aggregate China's total mackerel catch by year.

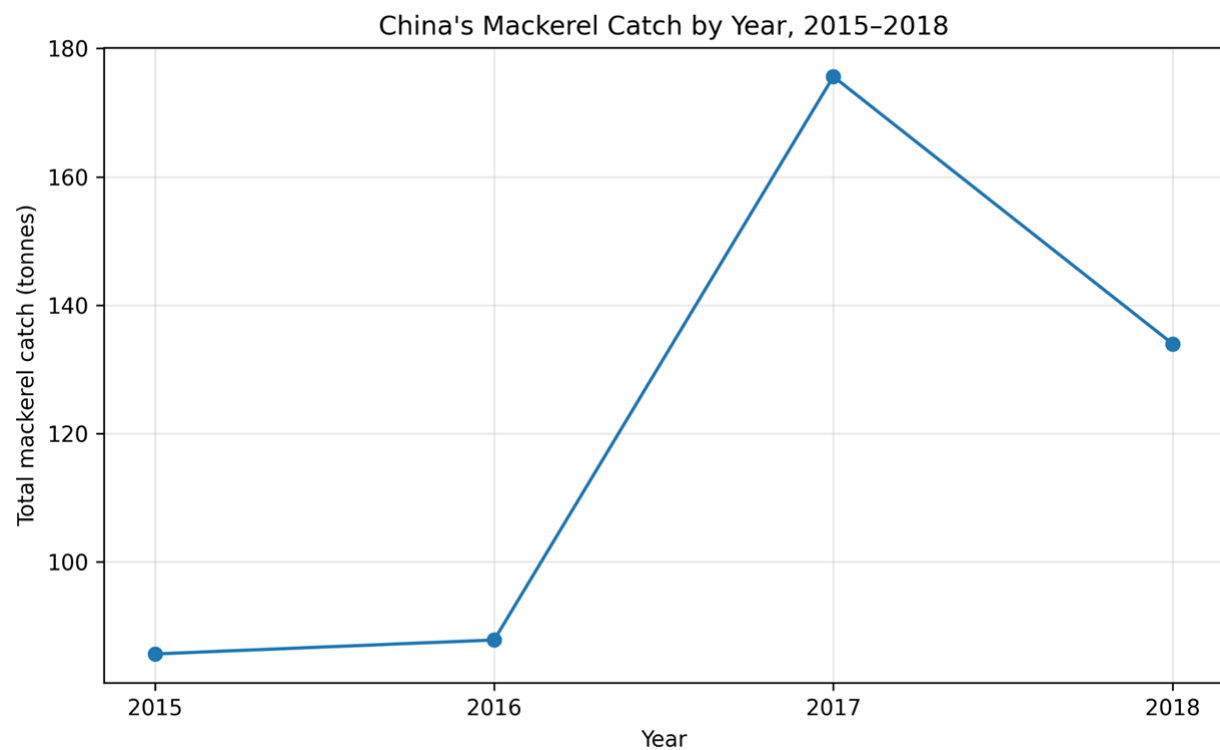


Figure 4.1.2 China's total mackerel catch from 2015 to 2018.

China trend insight

China's catch increased slightly from 85.66 tonnes in 2015 to 87.84 tonnes in 2016. It then almost doubled to 175.62 tonnes in 2017 before decreasing by approximately 23.7% to 133.99 tonnes in 2018. Even after the decline, the 2018 value remained substantially above the 2015 and 2016 levels.

4.2 Top 10 countries by total mackerel catch

```
SELECT fishing_entity AS Country,  
       CAST(SUM(tonnes) AS DECIMAL(18,2))  
       AS TotalMackerelsCatch  
FROM fishdb.data_source_ps3201ak  
WHERE common_name LIKE '%Mackerels%'  
      AND year > 2014  
GROUP BY fishing_entity  
ORDER BY TotalMackerelsCatch DESC  
LIMIT 10;
```

The screenshot displays the Amazon Athena Query Editor interface. On the left, the 'Data' panel shows the 'Data source' as 'AwsDataCatalog', 'Catalog' as 'None', and 'Database' as 'fishdb'. Below this, 'Tables and views' are listed, including 'data_source_ps3201ak', 'challenge', and 'mackerelcatch'. The main editor area contains a SQL query (Query 14) that selects the top 10 countries by total mackerel catch. The query is as follows:

```
1 SELECT fishing_entity AS Country,  
2       CAST(SUM(tonnes) AS DECIMAL(18,2)) AS TotalMackerelsCatch  
3 FROM fishdb.data_source_ps3201ak  
4 WHERE common_name LIKE '%Mackerels%'  
5      AND year > 2014  
6 GROUP BY fishing_entity  
7 ORDER BY TotalMackerelsCatch DESC  
8 LIMIT 10;
```

Below the query, the 'Query results' tab is active, showing a 'Completed' status. The results are displayed in a table with 10 rows and 2 columns: 'Country' and 'TotalMackerelsCatch'. The top 4 rows are visible:

#	Country	TotalMackerelsCatch
1	Fiji	8088.57
2	USA	3136.22
3	Viet Nam	858.87
4	Indonesia	783.58

Figure 4.2.1 Individual query used to rank the top 10 countries by total mackerel catch.

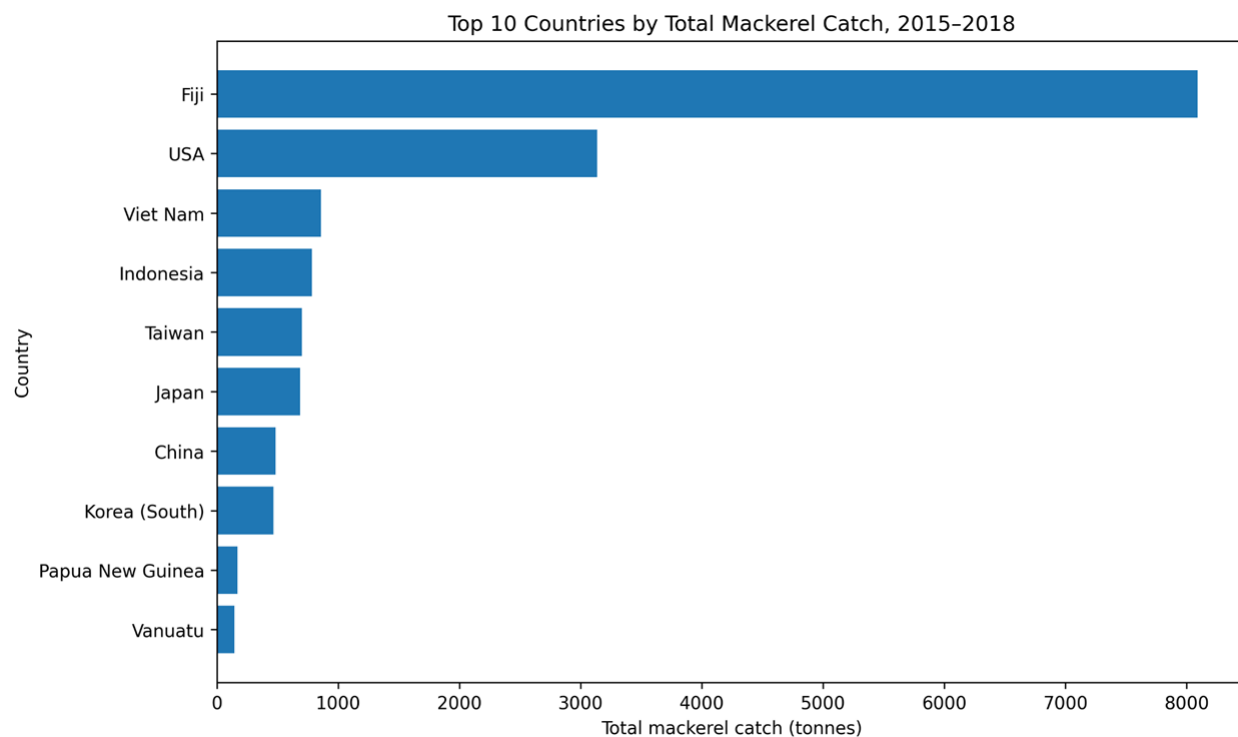


Figure 4.2.2 Top 10 countries by total mackerel catch from 2015 to 2018.

Cross-country insight

Fiji ranked first with 8,088.57 tonnes, followed by the USA with 3,136.22 tonnes. Fiji's total was approximately 2.58 times the USA total and represented about 52.1% of the combined catch among the top 10 countries. This indicates that mackerel catch in the selected data was highly concentrated in the leading country.

4.3 Fiji catch value against volume



The screenshot shows a SQL query editor window titled "Query 1". The query is as follows:

```
1 SELECT year,  
2     SUM(tonnes) AS TotalTonnes,  
3     CAST(CAST(SUM(landed_value) AS DOUBLE) AS DECIMAL(38,2)) AS TotalValueUSD  
4 FROM fishdb.data_source_ps3208a  
5 WHERE fishing_entity = 'Fiji' AND year > 2000  
6 GROUP BY year  
7 ORDER BY year ASC;
```

Figure 4.3.1 Query used for Fiji Catch Value against Volume

Results (18)					Co	
Q Search rows						
#	▼	year	▼	TotalTonnes	▼	TotalValueUSD
1		2001		77254.05771582389		134674527.76
2		2002		76195.93405928518		123456883.83
3		2003		73588.79789974239		135158609.74
4		2004		81312.2777414247		150703885.14
5		2005		79375.49119976751		150539943.92
6		2006		79635.07332670511		151637730.57
7		2007		81191.83578351403		157194049.45
8		2008		78513.92638274499		147269648.38
9		2009		77903.50532615832		152683764.90
10		2010		72733.47208935511		130256776.54
11		2011		77431.9185274237		138899886.70
12		2012		75233.0695537389		132931374.47
13		2013		70040.6767231242		120976404.88
14		2014		73397.9881186345		126764022.96
15		2015		73285.48552574217		127664640.71
16		2016		75632.52204672131		132542973.51
17		2017		82815.9769984194		156831382.97
18		2018		77148.0572644574		140581566.43

This section determines whether catch volume is a usable proxy for economic value. Figure 4.3.1) shows that the value is about twice as volatile as the volume and swings 21% in implied unit price at \$1,620–\$1,960/tonne. With the volume nearly flat over 2001 to 2018, we inferred that tonnage alone does not serve as an accurate indicator to sector revenue.

	count	mean	std	CV %	min	50%	max
TotalTonnes	18.00	76,816.11	3,395.80	4.42	70,040.68	77,201.06	82,815.98
TotalValueUSD	18.00	139,487,115.16	11,796,802.26	8.46	120,976,404.88	137,029,248.22	157,194,049.45
UnitPrice	18.00	1,813.47	88.51	4.88	1,620.26	1,808.03	1,959.91

Figure 4.3.1) Descriptive analysis of Fiji catch value and catch volume

This is further reinforced by a dual axes view of catch volume and landed value from 2001 to 2018.

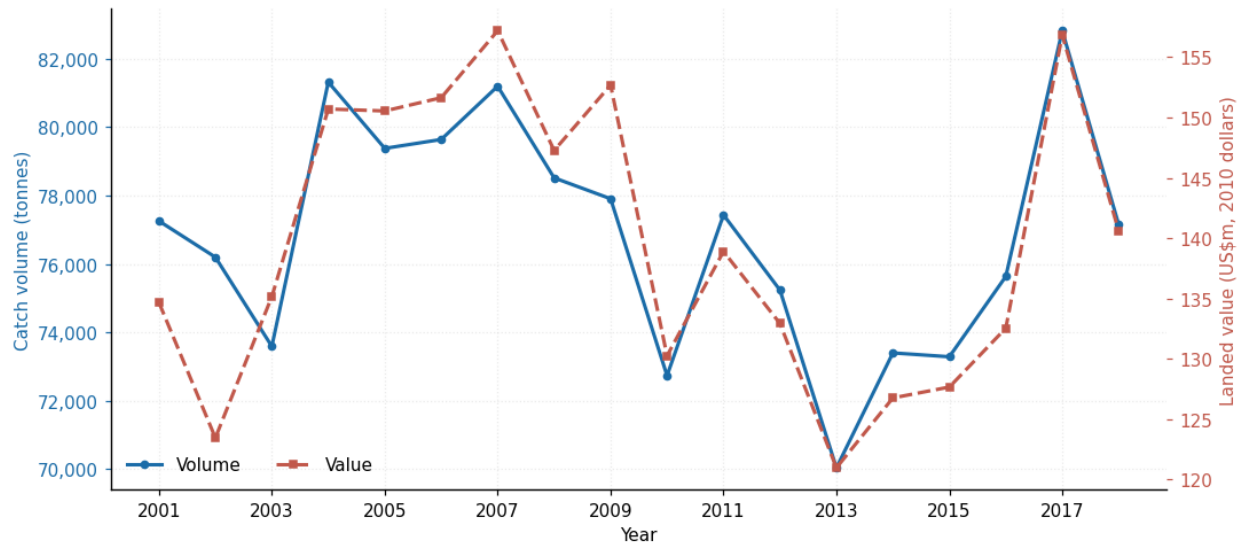


Figure 4.3.2) Dual-axis chart showing catch volume and landed value

However, we still cannot sufficiently represent the meanings of the divergence between the two quantities. Therefore, in Figure 4.3.1) we indexed the chart to accurately quantify the percentage points of divergence from the shared base in 2001.

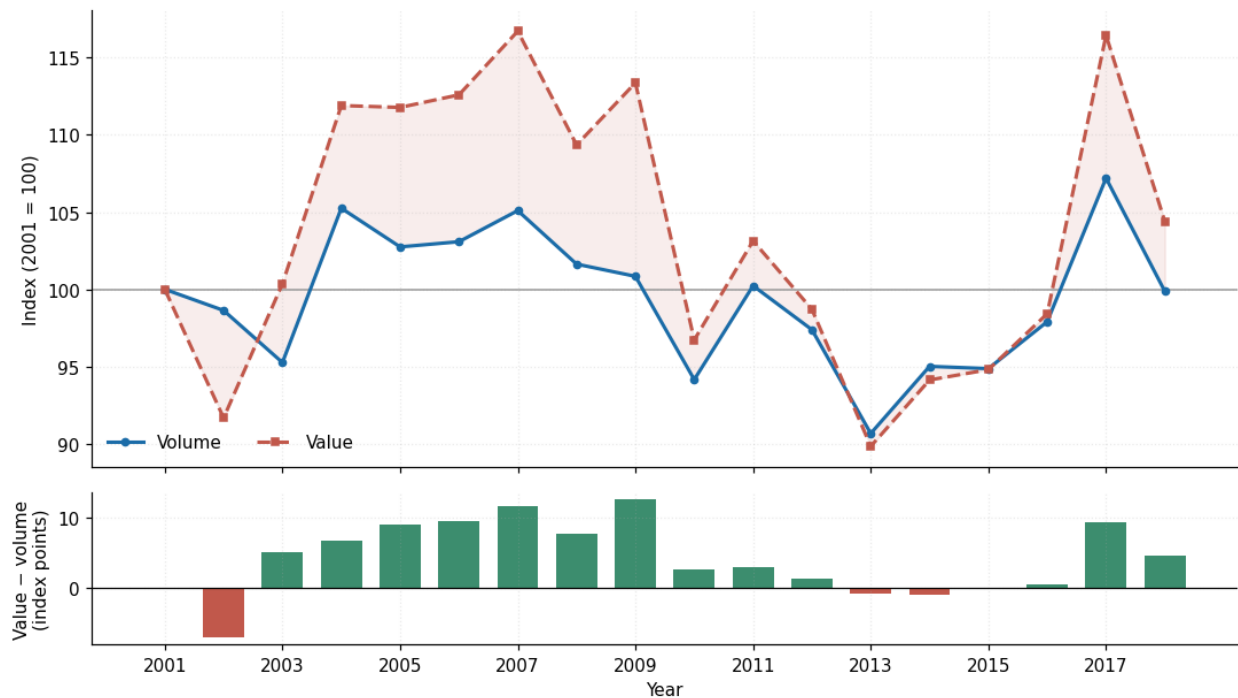


Figure 4.3.3) Dual-axis chart indexed to 100 in 2001

As seen in Figure 4.3.4), dividing value by volume strips out the quantity effect and leaves the composition effect: what a tonne of Fijian catch was worth, year by year, in constant 2010 dollars.

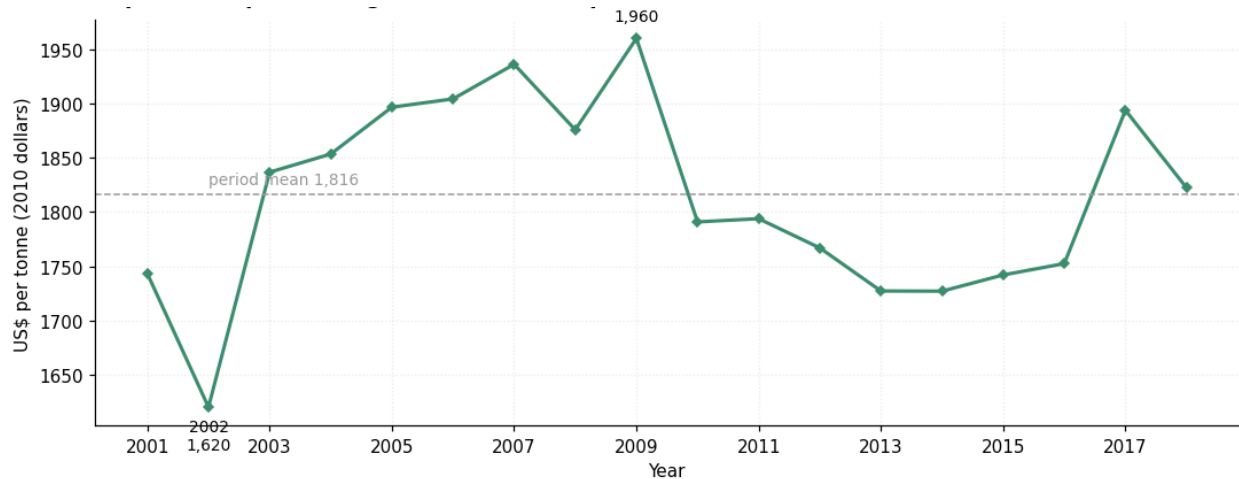


Figure 4.3.4) Implied unit price chart across the period (2001-2018)

The result showed that unit prices ranges from \$1,620 to \$1,960 per tonne with a spread of 21%. In Figure 4.3.5) we show that price and volume are not completely coupled using Ordinary Least Squares (OLS) regression ($R^2 = 0.804$), meaning volume explains only 80% of price variance. Due to the smaller dataset, we used Huber loss to reduce the sensitivity to outliers, resulting in a loss function that increases linearly with large residuals. We also compared it to Tukey loss where weight falls to exactly zero past $c \approx 4.685\hat{\sigma}$.

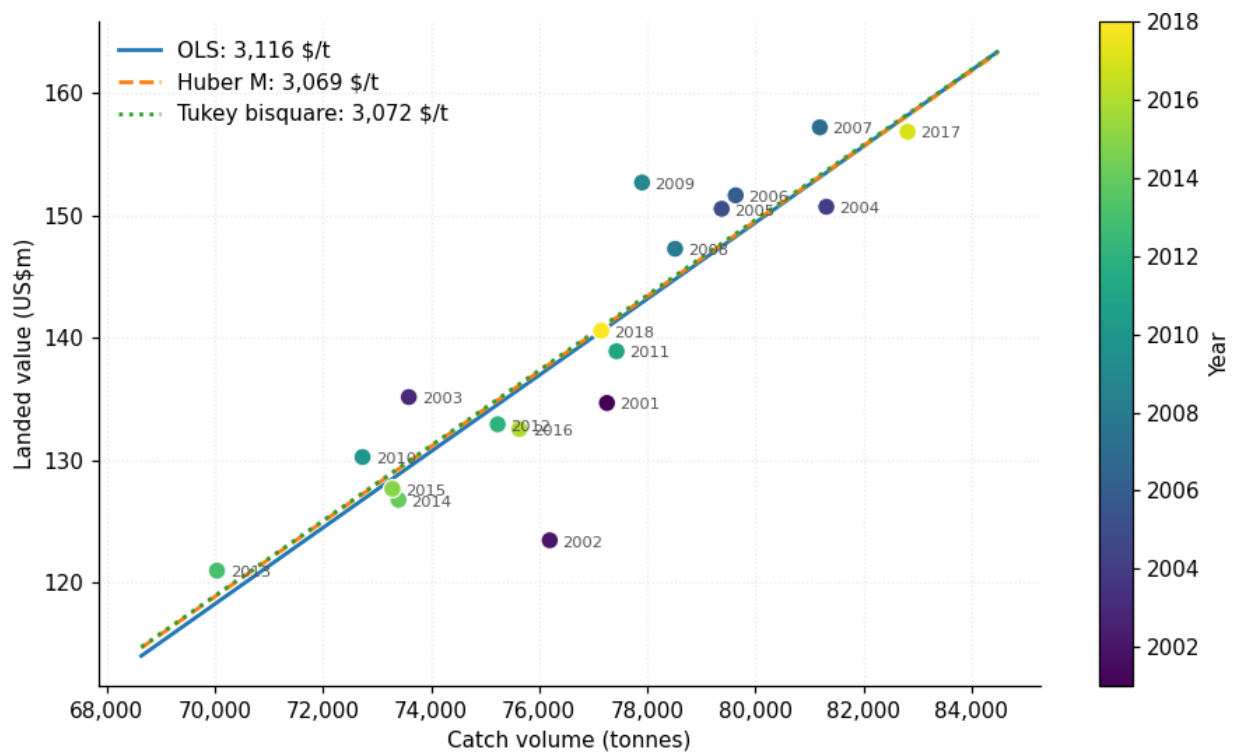


Figure 4.3.4) Marginal value per tonne rises with volume across the observed range

We also found that rising value per tonne of fish is correlated with at least three confounders that were not accounted for in the data: (i) the shift in species composition toward higher-value fish such as tuna; (ii) real price movement within a species mix that did not change; (iii) or a change in the Sea Around Us reconstruction method for that year.

Lastly, we found that there were no outliers (z -score > 3) on both volume and value with the largest deviation being 2.09 standard deviation in volume in 2017. The two most downweighted years by Tukey loss was 2002 ($w=0.27$) and 2009 ($w=0.59$).

4.4 Who fishes where (2015-2018)

Query 1 : X Query 2 : X

```

1 SELECT WhereCaught, Country, SUM(TotalWeight) AS OverallMackerelWeight
2 FROM fishdb.mackerelscatch
3 GROUP BY WhereCaught, Country
4 ORDER BY OverallMackerelWeight DESC;

```

SQL Ln 1, Col 1

Run again Explain L? Cancel Clear Create

Reuse query results up to 60 minutes ago

Query results Query stats

Completed Time in queue: 68 ms Run time: 1.048 sec Data scanned: 537.97 KB

Results (32) Copy Download results CSV

Search rows

#	WhereCaught	Country	OverallMackerelWeight
1	Fiji	Fiji	7974.3459800608
2	Pacific, Western Central	USA	3109.4816183456
3	Pacific, Western Central	Viet Nam	858.8742179707
4	Pacific, Western Central	Indonesia	783.5793809094

Figure 4.4.1) Query Used as well as the data below

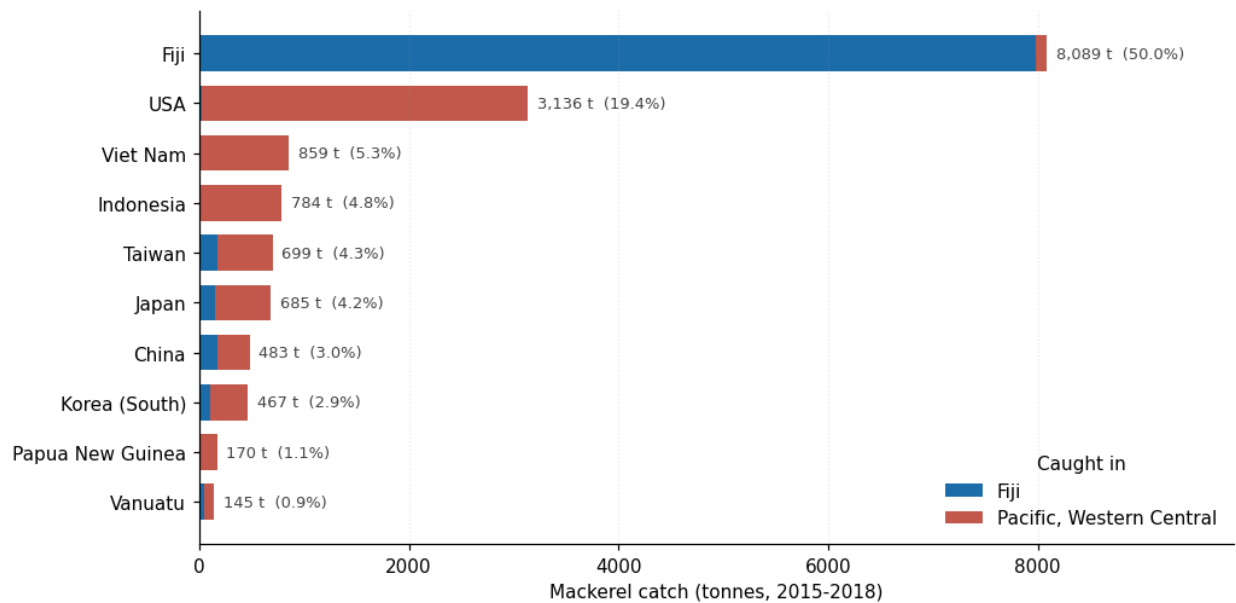
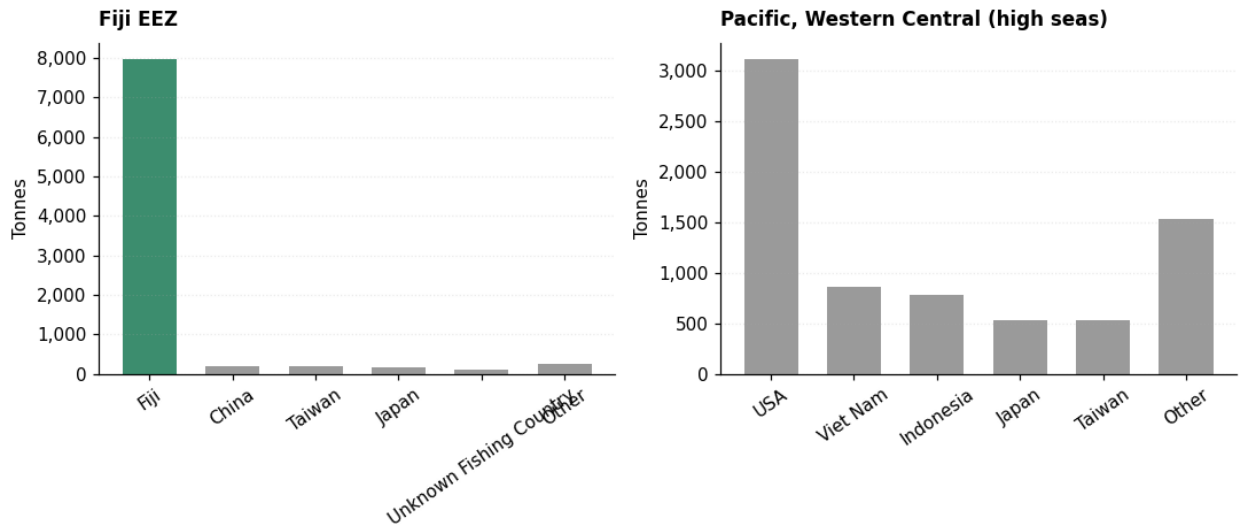


Figure 4.4.2) Tonnes of Mackerel caught by year by country in the Fiji EEZ



We inferred that Fiji is a near-monopolist and a marginal participant depending only on which side of the 200-nautical-mile line the catch occurred. Two limitations: (i) reconstructed nature of Sea Around Us dataset reports landings with estimated unreported and discarded catch and; (ii) the years are filtered.

4.5 Singapore High Seas Catch Volume(5 Decades)

Query 1 :  | Query 2 :  | Query 3 : 

```
1 CREATE OR REPLACE VIEW fishdb.singapore_fishing_trend AS
2 SELECT
3     year,
4     fishing_entity AS Country,
5     COALESCE(area_name, 'Open High Seas / Global') AS WhereCaught,
6     scientific_name,
7     common_name,
8     SUM(tonnes) AS TotalWeightTonnes,
9     CAST(CAST(SUM(landed_value) AS DOUBLE) AS DECIMAL(38,2)) AS TotalLandedValueUSD
10 FROM fishdb.data_source_ps3208a
11 WHERE fishing_entity = 'Singapore'
12 GROUP BY year, fishing_entity, area_name, scientific_name, common_name
13 ORDER BY year DESC, TotalWeightTonnes DESC
```

Fig 4.5.1 The Query used for creating view in Singapore Fishing Trend

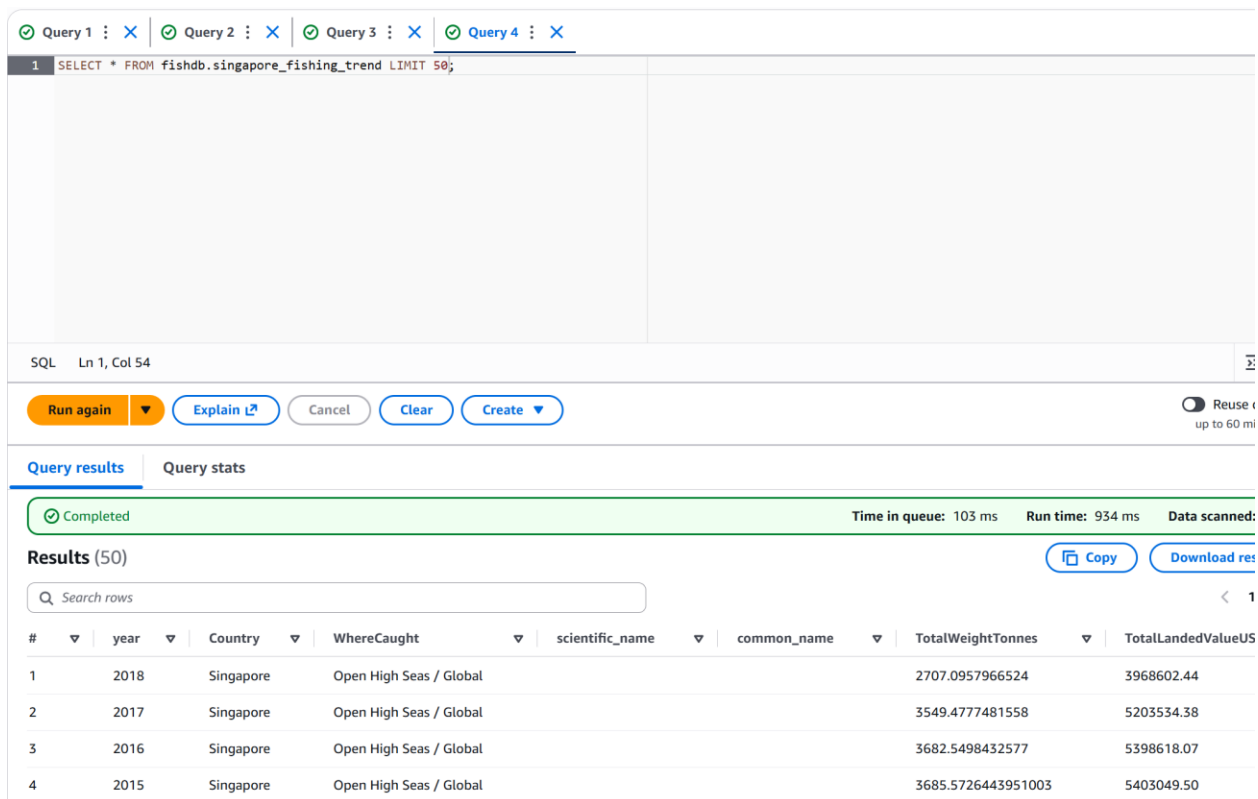


Fig 4.5.2 The Query used for obtaining data in Singapore Fishing Trend from 50 years

This section examines the overall trend of Singapore's High Seas catch and asks whether a single linear trend is sufficient to describe it. Figure shows that the catch volume peaked in 1984 before falling 92% by 2018. The log-scale panel is needed to confirm whether the rate of decline is roughly constant or accelerating.

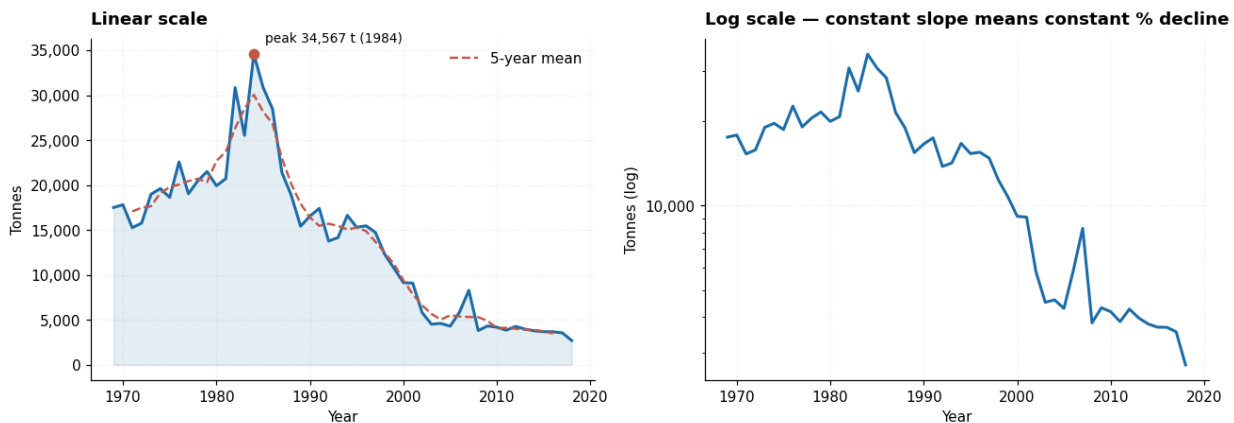


Figure 4.5.3) Linear and log scale of Singapore catch volume

The linear scale, the climb to 1984 peak and the subsequent fall both appears smooth and both the fall appears. To check if the percentage rate decrease is constant. The log scale panel resolves this, and the slope is not constant and it is far shallower in pre 1984 period rather than after this tells us that decrease in the rate accelerated once it hit its peak.

4.2. Data Interpretation and Limitations

(This part is for the above 2 – can change later)

- The analysis is descriptive: it identifies differences, peaks and trends but does not prove why the changes occurred.
- The dataset ends in 2018, so the results describe historical patterns and should not be presented as current fishing conditions.
- The MackerelsCatch view follows the exact grouping structure provided in the project brief. The individual queries used SUM at the year or country level to produce clearer analytical totals for visualisation.
- The graphs were generated from CSV files downloaded directly from successful Athena query results, reducing the risk of manual transcription errors.

6 Difficulties and Solutions

6.1 Difficulty finding real business insights

AWS identifies this as a core challenge of data visualisation: teams either oversimplify the data and lose the key information or introduce human prejudice by preselecting the data that suits the story they already wanted to tell or exaggerate by presenting unrelated series as a correlation (AWS, n.d.).

Early in the project we were guilty of the first and third. We could see that trade volume and trade value moved differently across the period, but our first draft either

flattened that into a single "trade is growing" line or implied a causal relationship we had no evidence for.

We solved this with an AI-assisted, human-verified workflow: (i) We collected academic and industry papers on seafood trade and price formation and used Claude to summarise them into digestible briefs; (ii) We then used the `/agency-agents data-visualisation-engineer` skill to interrogate the dataset and; (iii) Each claim was traced back to the underlying figures by a team member before it entered the report. Anything we could not reproduce from the raw data was cut.

6.2 Access to AWS Capstone Learner Lab environment

The environment would not permit us, and because the IAM policies, roles, and the IDE were pre-configured within that specific lab instance, there was no workaround available from our side. We sent our personal emails to the lecturer, and a clean environment was provided.

6.3 Time pressure in writing the report

With the analysis running late, we risked compressing the write-up into the final minutes—the point at which oversimplification and unchecked claims are most likely to slip through, undoing the verification work described above. We addressed this structurally: (i) split the work, delegated tasks and; (ii) had an earlier deadline before the official submission deadline to plan for any unexpected incidents with submission

7 Reflection

7.1 New technical knowledge gained

Parquet stores each column contiguously, so the engine reads only what the query names, and values of one type sitting next to each other compress far better than interleaved rows do. Abadi et al. quantify this at a factor of three or more on read-intensive analytical workloads, driven by late materialisation and compression [13]; Ivanov and Pergolesi's benchmarking confirms the format choice measurably changes performance, with Parquet under Snappy compression performing best on the SparkSQL stack [14] —

the same columnar-page arrangement Athena sits on. Our own pipeline is a direct, small-scale application of that literature: 561,675 rows, queries touching two or three columns, and a billing model that charges by bytes scanned.

Athena provisions nothing: there is no cluster to size, no instance to start, and the first query on a new table runs immediately.

7.2 Understanding of AWS services

Stage	Service	What it contributes	What it cannot do
Ingestion	Cloud9 → S3	A working environment with the AWS CLI already authenticated, and durable object storage that costs nothing to keep data in	S3 stores bytes. It has no idea what a column is. S3 Select can query <i>one</i> object and nothing more.
Catalogue	AWS Glue	Reads the Parquet footers, infers columns and types, and writes a table definition into the Data Catalog	Glue holds metadata only. No data moves, and no query can be run against Glue itself.
Query	Amazon Athena	Executes SQL against S3 objects using the Glue table definition; creates views; writes results to the second bucket	Athena has no storage of its own and no catalogue of its own. Remove either and it has nothing to query.

8 AI Tools and Prompts

8.1 AI use 1

Tool used: Google Gemini

Prompt: How do I check the countries that are available in the Global fishing data?

How the output was checked or used:

Option 1: Using Amazon Athena (Recommended if Glue Crawler ran)

Open your Amazon Athena query editor and run a `DISTINCT` query to list all unique countries/entities present in your table:

SQL



```
SELECT DISTINCT fishing_entity
FROM your_glue_table_name
ORDER BY fishing_entity ASC;
```

The following output was used to get the prompt to guide myself how to get the individual countries as I wanted to find out if Singapore was in the list.

8.2 AI use 2

Tool used: Give me ideas that I could possibly Query to analyse data. Do not give me the Query just yet. I want to try it myself.

Prompt: Give me ideas that I could possibly Query to analyse data. Do not give me the Query just yet. I want to try it myself.

How the output was checked or used: we tried Querying ourselves by referring to online resource material and asked AI to check for mistakes in my query to ensure best results.

1. Temporal & Trend Analysis

- **The Peak Decade:** Find out which specific 10-year period (e.g., between 1990 and 2000 vs. 2000 and 2010) had the highest cumulative fish catch globally.
- **Post-2000 Shift:** Compare the average annual catch tonnage of a specific country before the year 2000 versus the years 2000 and onward to see if their fishing activity increased or scaled back in recent decades.
- **Economic Inflation Check:** Track the average landed value per ton of fish year-over-year to see if the financial return per unit of fish caught has trended upward over time.

8.3 AI Use 3

Tool Used: Chatgpt

Prompt: Check and refine an Athena SQL query to calculate China's total mackerel catch by year.

Prompt: Create an Athena SQL query to identify the top 10 countries by total mackerel catch since 2015.

Prompt: Using the uploaded Athena CSV results, create a line graph comparing Fiji's open-seas, EEZ and combined catch values.

Prompt: Create a line graph showing China's total mackerel catch by year from 2015 to 2018.

Prompt: Create a horizontal bar chart showing the top 10 countries by total mackerel catch and identify the main trends.

8.4 AI Use 4

Tool used: Anthropic Claude, running the academic-paper-strategist and academic-paper-composer skills from the academic-paper-skills framework, used to structure and draft this report from the team's existing task logs and analysis files.

9 References

[1] Streit M, Gehlenborg N. Temporal data. *Nature Methods*. 2015;12(2):97-97 DOI: 10.1038/nmeth.3262

[2] Brath R, Hagerman C, Sorenson E. Why Two Y-Axes (Y2Y): A Case Study for Visual Correlation with Dual Axes. In: 2020 24th International Conference Information Visualisation (IV). IEEE; 2020:38-44 DOI: 10.1109/iv51561.2020.00016

[3] Schroeder JP. Bicomponent Trend Maps: A Multivariate Approach to Visualizing Geographic Time Series. *Cartography and Geographic Information Science*. 2010;37(3):169-187 DOI: 10.1559/152304010792194930

[4] Levontin P, Baranowski P, Leach AW, et al. On the role of visualisation in fisheries management. *Marine Policy*. 2017;78:114-121 DOI: 10.1016/j.marpol.2017.01.018

